# SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency

HAYLEY LEBLANC, The University of Texas at Austin, Austin, United States
NATHAN TAYLOR, Semgrep, Seattle, United States
JAMES BORNHOLT, The University of Texas at Austin, Austin, United States and Amazon Web Services Inc, Seattle, United States
VIJAY CHIDAMBARAM, The University of Texas at Austin, Austin, United States

This work introduces a new approach to building crash-safe file systems for persistent memory. We exploit the fact that Rust's typestate pattern allows compile-time enforcement of a specific order of operations. We introduce a novel crash-consistency mechanism, *Synchronous Soft Updates*, that boils down crash safety to enforcing ordering among updates to file-system metadata. We employ this approach to build SquirrelFS, a new file system with crash-consistency guarantees that are *checked at compile time*. SquirrelFS avoids the need for separate proofs, instead incorporating correctness guarantees into the typestate itself. Compiling SquirrelFS only takes tens of seconds; successful compilation indicates crash consistency, while an error provides a starting point for fixing the bug. We evaluate SquirrelFS against state-of-the-art file systems such as NOVA and WineFS, and find that SquirrelFS achieves similar or better performance on a wide range of benchmarks and applications.

CCS Concepts: • **Software and its engineering → File systems management**; **Software verification and validation**; • **Information systems → Storage class memory**; • **Computer systems organization → Reliability;**

Additional Key Words and Phrases: Crash consistency, file systems, persistent memory, rust, static bug detection

**ACM Reference Format:**
Hayley LeBlanc, Nathan Taylor, James Bornholt, and Vijay Chidambaram. 2025. SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency. *ACM Trans. Storage* 21, 4, Article 27 (November 2025), 39 pages. https://doi.org/10.1145/3769109

## 1 Introduction

One of the most important properties for file systems is to preserve their integrity and user data in the face of a crash or a power loss [7, 12, 27, 30, 45, 46, 53]. Unfortunately, building crash-consistent file systems is challenging; checking or ensuring crash consistency is even more so [9, 41].

This work was supported by NSF CAREER #1751277, NSF CCF #2124044, and donations from Amazon, Toyota, and VMware.
Authors' Contact Information: Hayley LeBlanc (corresponding author), The University of Texas at Austin, Austin, Texas, United States; e-mail: hleblanc@utexas.edu; Nathan Taylor, Semgrep, Seattle, Washington, United States; e-mail: ntaylor@cs.utexas.edu; James Bornholt, The University of Texas at Austin, Austin, Texas, United States and AmazonWeb Services Inc, Seattle, Washington, United States; e-mail: bornholt@cs.utexas.edu; Vijay Chidambaram, The University of Texas at Austin, Austin, Texas, United States; e-mail: vijayc@utexas.edu.

ACM 1553-3077/2025/11-ART27
https://doi.org/10.1145/3769109

Table 1. Comparison of Crash-Consistency Approaches

| Approach | Complete | Dev effort | Time to check |
|---|---|---|---|
| Testing [35, 37, 42, 49] | No | Low | Medium |
| Verification [11, 28] | Yes | High | High |
| This work | Yes | Medium | Low |

The table compares the two main existing crash-consistency techniques, testing and verification, with this work. This work strikes a balance between completeness, development effort, and the additional time required to check for bugs.

There are two main approaches to building file systems today, as summarized in Table 1. First, we build file systems using low-level languages like C, and we use runtime testing to gain some confidence in the correctness of the systems [35, 36, 42, 49, 50, 67]. Note that this approach is necessarily incomplete; testing can only reveal bugs, not prove their absence. However, this approach allows rapid development, and entire testing ecosystems have sprung up around this basic approach, like the widely-used xfstests [63] and Linux Test Project [44].

A different approach to building file systems is to verify them: we write a high-level specification of correct behavior (including crash behavior) and then prove that the implementation matches the specification [9–11, 28, 57]. This approach can prove that the implementation does not have certain classes of bugs; however, it comes at a high cost. For each line of code in the implementation, we may need to write 7–13 lines of proof. Writing and maintaining proofs is time-consuming and requires specialized expertise, constraining rapid development.

In this work, we seek to find a middle ground between these two approaches. We would like to verify some aspects of file systems, but without the burden of having to write and maintain proofs. In particular, we are interested in *crash consistency*, a correctness property that is especially difficult to test for. In order to be crash consistent, systems must ensure that updates become persistent on storage media *in the correct order*; however, hardware or caching layers may reorder updates to improve performance in unanticipated ways. Exposing crash-consistency bugs thus requires one to find and reproduce these low-level orderings, which requires specialized testing software [35, 36, 42, 49, 50, 67]. Our goal is to develop lightweight approaches to statically check for crash-consistency bugs without the overhead of full verification.

We exploit two recent developments to achieve this goal (Section 2). First, the Rust programming language has a strong type system that supports powerful compile-time safety checks. Our work takes inspiration from Corundum [31], a Rust crate (library) that uses Rust's type system to check low-level PM safety properties. In this work, we observe that Rust's type system can also statically enforce that certain operations are carried out in a given order [26, 38]. Since the root of crash consistency is ordering updates to storage, if we can encode those ordering-based invariants in the type system, the compiler can ensure the invariants hold at compile time.

However, to do so, crash consistency must be derived purely from ordering-based invariants; some mechanisms such as journaling use writes to a log to obtain atomicity, which is harder to encode in the type system. Soft updates achieves crash consistency purely via ordering [46], but the traditional soft updates scheme is complex and hard to implement [3, 24].

We observe that the low latency of persistent memory [64, 67] allows file-system operations to be *synchronous*; all updates to storage media are durable by the time each operation returns [21, 33, 34, 39, 65]. We take advantage of persistent memory's synchronous updates and byte addressability to develop a new mechanism for crash consistency, which we term *Synchronous Soft Updates*.

Synchronous Soft Updates builds on the classical soft updates mechanism [46], but avoids most of the complexity that prevented the widespread adoption of soft updates [3]. Two of the most complicated aspects of soft updates, dependency structures and cyclic dependency

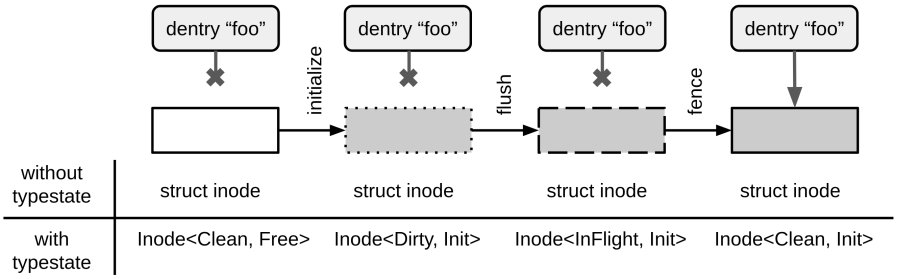


Fig. 1. For a soft updates file system to be crash-consistent, directory entries should only point to fully-initialized, durable inodes. In existing file systems, all persistent inodes have the same type, regardless of whether they are durable or have been initialized. With typestate, durability and the inode's contents are reflected in its type.

management, arise due to the need to track ordering requirements between block-sized updates across asynchronous operations. Synchronous Soft Updates eliminates these challenges entirely by using fast, fine-grained storage to back synchronous operations.

We ensure that the ordering invariants of Synchronous Soft Updates hold by using the Rust compiler. We take advantage of Rust's support for the *typestate pattern*, an API design pattern where an object's type reflects the operations that have been performed on it [58]. The legal order of operations is encoded in function signatures and enforced by Rust's typechecker. For example, an uninitialized inode has a different type than an initialized one; attempting to use one where the other is expected will result in a compile-time error. Figure 1 illustrates the approach.

We implement Synchronous Soft Updates in a new file system for PM called SquirrelFS and use the typestate pattern in Rust to check that update orderings are implemented correctly. SquirrelFS provides crash-atomic metadata system calls, including `rename`; on the original soft updates, a crash during rename could result in both the source and destination existing. SquirrelFS assumes that the CPU caches are volatile and that cache lines must be flushed or written back and ordered using store fences to ensure durability. SquirrelFS compiles and typechecks in seconds, whereas running verification on existing storage systems takes minutes or hours. Building SquirrelFS required no modifications to the Rust language.

We evaluate SquirrelFS by comparing to a number of file systems meant for persistent memory, such as NOVA [65] and WineFS [33] (Section 5). We use Intel's Optane DC Persistent Memory Module for our comparison, and find that SquirrelFS offers comparable or better performance to other PM file systems across a broad range of workloads. The current SquirrelFS prototype prioritizes simplicity of update ordering rules over performance in some areas, leading to relatively high mount times and memory utilization; however, these are not fundamental limitations of the design. We also model the design of SquirrelFS using the Alloy model-checking language [32] to gain confidence in the correctness of its Synchronous Soft Updates mechanism.

We note that SquirrelFS is not fully verified, and thus does not obtain the strong correctness guarantees of verified storage systems like FSCQ [11]. Crash-consistency bugs may still occur in SquirrelFS if their root causes are unrelated to ordering, if the ordering rules enforced by the compiler are incorrect, or if trusted code in SquirrelFS's implementation or the Rust compiler are buggy. For example, SquirrelFS's ordering rules guarantee that inodes are always initialized before they are linked into the file system tree, but they do not guarantee that the contents of the inode are correct. SquirrelFS's static checks are also limited by the capabilities of the Rust compiler. For instance, the Rust compiler cannot check properties about variable-sized sets of data structures, as checking such properties is undecidable in general.

SquirrelFS offers a useful new point in the spectrum of approaches to building robust storage systems; it provides weaker guarantees than verified systems, but comes at a lower cost. As such, we hope that it proves useful for developers of storage systems that require strong guarantees, good performance, and rapid development.

In summary, this work makes the following contributions:

- Statically-checked crash consistency, an approach where high-level properties are encoded into the type system and checked at compile time (Section 3)
- The Synchronous Soft Updates crash-consistency mechanism for persistent-memory file systems (Section 3.1)
- The SquirrelFS prototype, along with a discussion of lessons learned during its development (Section 4).

SquirrelFS and its Alloy model are publicly available at https://github.com/utsaslab/squirrelfs.

## 2 Background and Motivation

### 2.1 Crash Consistency

A file system is *crash consistent* if it can recover to a consistent state after a power loss or a crash [7, 12, 53]. A consistent file system is one where all the metadata is in sync; for example, two files cannot (mistakenly) claim the same data block. Files present before the crash must exist post-crash, and the data in files must remain valid.

**Crash-consistency mechanisms**. Crash consistency is generally achieved using mechanisms such as journaling [27, 51], copy-on-write [8, 30, 45], or soft updates [46]. The root of crash consistency is correctly *ordering* writes to storage [12]; for example, a data block must be initialized before a file points to it. Soft updates achieves crash consistency by carefully ordering in-place updates to storage such that all possible crash states are consistent [46]. To enforce ordering, soft updates must track updates across asynchronous operations and resolve cyclic dependencies when they arise. Though soft updates is used in FreeBSD [47], it has not been widely adopted due to its high complexity.

**Ensuring crash consistency**. Ensuring that a given file system achieves crash consistency is challenging. There are two main approaches. The first approach is testing, in which possible crash states of a file system are explored and checked for consistency. Obtaining these crash states requires support from tools like eXplode [67], CrashMonkey [49], Hydra [36], Chipmunk [42], or Vinter [35]. While such testing tools can find many bugs, they cannot prove overall correctness or the absence of crash-consistency bugs.

The second approach is to build verified file systems. A developer writes a high-level specification of correctness and a lower-level implementation, and proves that the implementation satisfies the specification. This approach is stronger than testing in that it can prove strong correctness properties and verify that there are no bugs. However, it comes at a high cost: the developer has to write 7–13 lines of proof for every line of code. For example, BilbyFS [2] required 13k lines of proof for 1k lines of implementation code; VeriBetrKV [28] used 45K lines of proof for 6k lines of implementation. Another verified file system, FSCQ [11], has interleaved proof and implementation code that is 10× the size of the most similar unverified system.

This heavy proof burden constrains development in a number of ways. First, building a verified system requires proof-writing expertise, which restricts the set of developers who can work on it. Second, proofs must be written in tandem with the code that they verify, which extends development time. Finally, making changes to the system requires corresponding changes to the proofs, making maintenance slow and preventing rapid updates.

```
fn new_file() {
    // Dentry<Free>
    let d = Dentry::get_free_dentry();
    // Inode<Free>
    let i = Inode::get_free_inode();
    // Dentry<Init>
    let d = d.set_name("foo");
    let d = d.commit_dentry(i);
                              ^ expected `Inode<Init>`, found `Inode<Free>`
}
```

Listing 1. The listing shows the typecheck process throwing an error when an uninitialized inode is passed to a function that expects an initialized inode.

Corundum [31] is a Rust crate for PM systems that, like SquirrelFS, uses the Rust type system to enforce certain low-level PM safety properties at compile time. For example, Corundum ensures that every update to PM occurs in a logged transaction, and prevents the storage of pointers to volatile memory in durable structures. SquirrelFS was inspired by Corundum and aims at enforcing higher-level properties like file-system crash consistency with Rust.

### 2.2 The Opportunity: Rust and PM

We observe an opportunity to ensure file-system crash consistency in a cheap manner.

First, we note that the Rust programming language can **statically enforce a specific order on operations** via its support for the *typestate pattern* [5, 26]. Briefly, the typestate pattern enables an object's runtime state to be encoded in its type [58]. This state can be checked at compile time via typechecking, ensuring that a given operation can only occur on a specific type. Typestate information is stored in zero-sized types that incur no runtime overhead.

For example, one consistency rule enforced by soft updates is that a directory entry should never point to an uninitialized inode. Listing 1 shows how typestate is used to enforce this rule. To create a new file, we first obtain a free directory entry and inode. Initially, both objects have typestate `Free`. Then, we initialize the directory entry, transitioning its type to `Dentry<Init>`. The listing then has a bug in which the directory entry's inode number is set by `commit_dentry()` before the inode is initialized, breaking the consistency rule. The Rust compiler catches this bug because the inode's current typestate `Free` does not match the typestate `Init` expected by the function.

Since soft updates is entirely built on ordering updates to file-system objects, we can translate the required partial order into a set of types and use Rust's type checking to enforce the order. Thus, the invariants we want to maintain are translated into something the type system and compiler can enforce. We note that we are able to do this with an *unmodified* Rust compiler; the new types introduced are no different to the compiler from existing types in the codebase.

However, implementing soft updates correctly remains challenging even with typestate support. With soft updates, file-system updates are applied to the page cache in DRAM, and then later written to storage in the right order. Determining the right order requires tracking complex dependencies across asynchronous operations. When a single file-system metadata object (such as an inode or a bitmap) is updated multiple times, it can lead to cyclic dependencies.

This leads to our second observation: **persistent memory** (**PM**) file systems support synchronous operations thanks to the low latency of the storage media [64, 66]. These file systems write updates directly to storage without first caching them in DRAM [21, 33, 34, 39, 65]. A **synchronous implementation** of soft updates for persistent memory **eliminates** the

complexities of asynchronous dependency management, greatly simplifying the mechanism and allowing the relevant invariants to be encoded in Rust's type system.

## 3 SquirrelFS

We now present the design and implementation of SquirrelFS, a novel file system that uses the unmodified Rust compiler to check its crash consistency. If the compilation is successful, it indicates that the ordering-based invariants hold throughout the file system: in other words, the checking is complete. If compilation fails, the error reported by Rust is useful in figuring out which operations are out of order. Compilation takes only seconds, offering quick feedback to developers.

SquirrelFS is built on two key ideas:

— A novel crash-consistency mechanism, Synchronous Soft Updates, that achieves crash consistency purely via ordering file-system operations (Section 3.1)
— Using the Rust typestate pattern to encode ordering invariants into the Rust type system (Section 3.2)

It is important to note that we are not modifying the Rust compiler in any way. To the Rust compiler, it is no different from type-checking any other code base; we are merely using the type checking to ensure that crash consistency holds in the file system.

We now describe the key ideas in more detail.

### 3.1 Synchronous Soft Updates

We develop ***Synchronous Soft Updates*** (**SSU**), a novel crash-consistency mechanism. SSU is based on the traditional soft updates approach, but differs in two key aspects. First, soft updates was designed for asynchronous settings, but all operations are synchronous in SSU. Second, soft updates does not provide atomic rename; a crash during a rename of `src` to `dst` can result in both being present after a crash. SSU fixes this flaw; renames are atomic, and a crash during rename will result in either `src` or `dst` after recovery.

We now discuss why we developed SSU, its key aspects, and how renames are atomic in SSU.

**Why a new mechanism?** To go with our overall approach of encoding ordering-based invariants into the Rust type system, we needed a mechanism that achieves crash consistency purely via ordering file-system updates. This rules out mechanisms such as journaling and copy-on-write that use writes to a log or an extra copy to obtain atomicity. Soft updates [46] obtains crash consistency by enforcing ordering on in-place persistent updates to file-system objects; thus, it was a good match. However, traditional soft updates suffered from two problems that we needed to tackle. The first challenge was that soft updates had significant complexity arising from needing to track dependencies between asynchronous file-system operations; the presence of cyclic dependencies also requires complex roll-back and roll-forward logic. The second challenge is that soft updates does not provide atomic operations, particularly rename; atomic rename is a crucial primitive for a number of POSIX applications [54]. Thus, we need to modify soft updates to tackle both its high complexity and lack of atomic operations.

**Synchronous operations**. We observe that the root of complexity in soft updates (such as cyclic dependencies and structures for tracking dependencies) is *asynchrony*. A *synchronous* implementation of soft updates neatly avoids these complexities. All updates would be made durable by the end of each system call, which would eliminate the need to track cross-operation dependencies. Cyclic dependencies would no longer arise because there are no pending updates that can conflict with each other. The SoupFS [20] soft updates file system for persistent memory eliminated cyclic dependencies using fine-grained updates, but still required asynchronous dependency tracking. A synchronous implementation is necessary to overcome both sources of complexity.

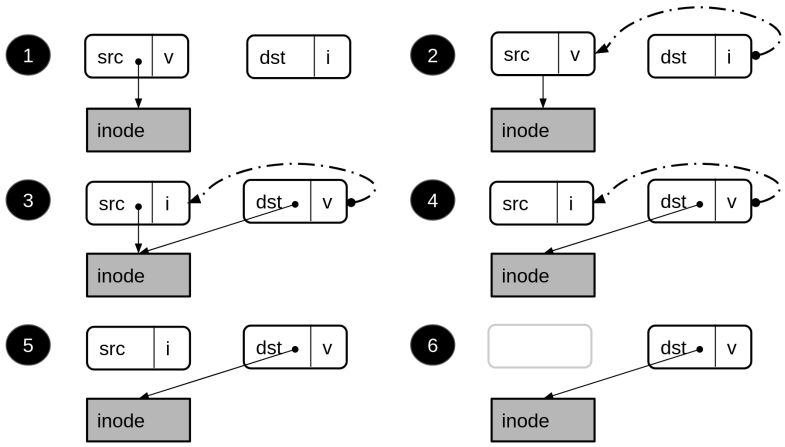


Fig. 2. The figure shows the steps in atomic soft updates `rename`. The dotted lines represent rename pointers and the solid lines represent inode pointers. Rounded rectangles represent directory entries. The labels "v" and "i" indicate whether a directory entry is valid or invalid, respectively. These labels are included for clarity only; validity is determined using a combination of the directory entry's inode number and, if set, an associated rename pointer.

A synchronous version of soft updates was not feasible until now, as running this on magnetic hard drives or even solid state drives would be prohibitively slow. However, synchronous soft updates is a good match for PM due to its low latency; system calls in many existing PM file systems are already synchronous [21, 33, 34, 65].

Similar to traditional soft updates, SSU maintains crash consistency by enforcing ordering among updates to file-system objects. SSU implements the original soft updates rules [25]:

(1) Never point to a structure before it has been initialized;
(2) Never re-use a resource before nullifying all previous pointers to it;
(3) Never reset the old pointer to a live resource before the new pointer has been set.

These rules are significantly easier to enforce in a synchronous setting, as there is no need to track dependencies across asynchronous operations. Like soft updates, SSU focuses on the integrity of file system metadata and cannot guarantee that operations on file data are atomic. SSU could be combined with journaling or copy-on-write to obtain stronger data guarantees.

**Atomic rename in SSU**. SSU ensures renames are atomic by cleaning up file-system state after a crash. In traditional soft updates, if there is a rename from `src` to `dst`, it is impossible to tell after a crash whether `src` or `dst` should be removed. To resolve this, SSU adds an extra field, called the *rename pointer*, to directory entries in order to persistently save enough information to complete the rename operation after a crash. The rename pointer in the destination directory entry points to the physical location of the source directory entry. The rename pointer allows the file system to follow soft updates rule 3 (never reset the old pointer before the new one has been set) while also retaining the ability to distinguish between `src` and `dst` after a crash.

Note that this is similar to what journaling-based file systems do; they write a log entry specifying `src` and `dst` so that the right clean-up action can be performed. In SSU, the information in this log entry is distributed over the source and destination inodes; taken together, they provide enough information to the file system.

Figure 2 illustrates the process. Step ❶ shows an example system state prior to the `rename` operation. In ❷, `dst`'s rename pointer (dotted line) is set to `src`. `dst` is invalid, and `src` is still valid.

In ❸, we make `dst` valid; this also logically invalidates `src`. This is an atomic point; after this step, the file system will always complete the rename operation. If the file system crashes prior to this step, the rename pointer is cleared on recovery. In ❹, we physically mark `src` as invalid. In ❺, the rename pointer is cleared, and in ❻ `src` is fully deallocated. Each step either modifies metadata that is invisible to the user (e.g., deallocating an orphaned directory entry) or atomically modifies a single 8-byte value. All modifications must be durable before proceeding to the next step.

**Rename recovery.** A question that arises is how the file system finds `src` and `dst`. This is an example of how SSU is tailored for PM file systems. In PM file systems, it is common for the file system to scan persistent objects to construct indexes in DRAM; we add the rename-recovery procedure into this scan. For most operations, recovering from a crash only requires clearing orphaned objects as they are encountered during this scan (Section 3.4). Rename recovery requires several additional steps to safely clear leftover rename pointers so that they do not interfere with subsequent rename operations.

A pseudocode Rust implementation of the rename recovery procedure is shown in Listing 2. For simplicity, this code is written without typestates. We describe the typestates used in `rename` and its recovery in Section 3.5. During both standard and post-crash remount, SquirrelFS first scans inodes and page descriptors to create an index mapping live inodes to their pages. After a crash, SquirrelFS passes this index to `rename_recover` (line 1 of Listing 2) and scans all directory entries in live directory pages (lines 2 and 3). For each directory entry, we first check if its rename pointer is set (line 4). If it is not set, then the directory entry was not the destination in an interrupted and incomplete `rename`. It may still require cleanup if it is the source for an interrupted `rename`, in which case it will be handled when we scan the corresponding destination. If the rename pointer is set, we know that this directory entry was the destination of a `rename` that crashed in step ❷, ❸, or ❹. We can now use the inode pointers of the source and destination directory entries to determine what steps are necessary for cleanup. If they do not point to the same inode, we either crashed in Step ❷ or ❹. Crash recovery is the same in both cases and only requires clearing the rename pointer (line 10). If we had crashed in Step ❷, this will effectively roll back to before the operation; if it was Step ❹, this will transition the system to Step ❺. Otherwise, if the two directory entries point to the same inode, we must have crashed in Step ❸. In this case, we recover by picking up where the `rename` operation left off and complete it using the same functions used for this part of `rename` during normal operation (lines 13-15). This process may leave orphaned structures, which will be cleaned up later in recovery.

### 3.2 Using Rust to Enforce Ordering

Rust's typestate pattern can be used to ensure that *a set of functions are always called in certain partial order*. A total order is not necessary, as many operations involve independent updates that can be safely reordered. As we discussed previously (Section 2), an object's typestate is encoded in generic type parameters in its definition, and the partial order is encoded in the function signatures of its associated functions.

We encode two states (as different type parameters) in the types of persistent objects:

- *Persistence* typestate is a representation of whether an object's most recent update(s) have been made durable. We use three persistence typestates: `Dirty`, `InFlight`, and `Clean`.
- *Operational* typestate represents the operations that have been performed on an object and is used to determine what operations can happen next.

Persistence and operational typestate are separate to capture the fact that most storage devices do not synchronously flush updates. For example, in persistent memory, updates go to the CPU caches first, and must be explicitly flushed to the persistent media.

```
fn rename_recover(pages: &PageIndex) -> Result<()> {
    for page in pages.filter(|p| p.get_type() == DIR) {
        for d in pages.dentries() {
            if d.rename_ptr().is_set() {
                // d was a dst dentry in an interrupted rename and
                // we crashed at step 2, 3, or 4.
                let src = d.get_src_from_rename_pointer();
                if d.get_ino() != src.get_ino() {
                    // we crashed at step 2 or 4
                    d.clear_rename_pointer().flush().fence();
                } else {
                    // we crashed at step 3
                    src.clear_ino().flush().fence();
                    d.clear_rename_pointer().flush().fence();
                    src.dealloc_dentry().flush().fence();
                }
            } // else, no cleanup needed for now
    }}}
```

Listing 2. The listing shows the recovery procedure to clean up interrupted `rename` operations.

```
impl Inode<Clean, Free> {
    fn init_inode(self, ino: u64, ...) -> Inode<Dirty, Init> {...}
}
impl Dentry<Clean, Alloc> {
    fn commit_dentry(self, inode: Inode<Clean, Init>)
        -> Dentry<Dirty, Committed> {...}
}
impl<S> Inode<Dirty, S> {
    fn flush(self) -> Inode<InFlight, S> {...}
}
impl<S> Inode<InFlight, S> {
    fn fence(self) -> Inode<Clean, S> {...}
}
```

Listing 3. Pseudocode implementations of file system objects with persistence and operational typestate. Typestate arguments are shown in bold.

Listing 3 shows implementations of several methods of persistent `Inode` and `Dentry` types with persistence and operational typestate as generic type parameters. The methods `flush` and `fence` invoke a cache line write back and store fence respectively and are generic with respect to operational typestate. These methods must be used to ensure updates are persistent before continuing; for example, `commit_dentry()` requires an `Inode<Clean, Init>` to ensure the inode's initialization cannot be transparently reordered with the directory entry updates.

This formulation of persistence typestate has several performance benefits. First, because the `flush` and `fence` methods can only be called on an object whose typestate indicates it is not

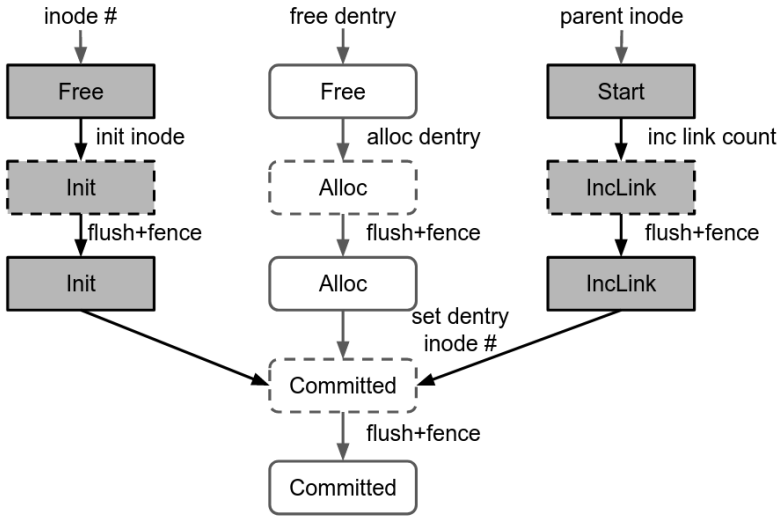


Fig. 3. The figure shows the persistent updates and corresponding dependencies made during `mkdir`. Inodes are dark gray rectangles and directory entries are white rounded rectangles. Each object is labeled with its operational typestate and its outline indicates whether it is clean (solid) or dirty (dotted).

yet persistent, typechecking will prevent redundant persistence operations (thereby improving performance). Second, developers can wait to flush updates until it is strictly necessary and can write additional transitions to enable multiple updates to share a single fence.

**Why Rust?** In order to obtain useful compiler-checked guarantees from the typestate pattern, each object must have exactly one typestate [58]. Thus, languages with unrestricted aliasing (e.g., C) cannot support the typestate pattern, as different aliases for the same value can have different types. Rust supports the pattern via its ownership type system, which ensures that each value has exactly one owner (and thus exactly one type).

### 3.3 Examples

To illustrate the typestates and dependency rules used in SSU, we describe two example operations, `mkdir` and `unlink`. The typestates shown in these examples are described in more detail in Table 2.

**Example 1:** `mkdir`. We first describe the dependency rules in an SSU implementation of `mkdir`, shown in Figure 3. To be crash consistent, an SSU implementation of `mkdir` must ensure (1) that a structure never points to an uninitialized resource, and (2) that each inode's link count is greater than or equal to its actual number of links. Both rules prevent dangling links in the event of a crash.

During `mkdir`, three file-system objects are modified: an inode for the new directory, a directory entry for the new directory, and the inode of the parent directory. In Figure 3, inodes are represented by dark gray boxes and directory entries are represented by white boxes. Each durable object is labeled with a typestate representing its current status at each point during the operation. The full set of typestates and dependencies for each durable structure are shown in Figure 8 for inodes and Figure 9 for directory entries. The initial updates to each data structure are independent, so there are no dependencies between them and they can share a single store fence to ensure durability before setting the directory entry's inode number (not shown). SquirrelFS uses volatile allocation structures, so it does not durably store any allocator-related metadata.

The system first finds the parent inode and obtains a free directory entry in one of the parent's pages as well as a free inode. The inode is then initialized (i.e., setting its inode number, link count, timestamps), the directory entry's name is set, and the parent inode's link count is incremented.

Next, we commit the directory entry by setting its inode number. This makes the directory entry valid and connects the inode to the file system tree. Directory entry commit is dependent on inode and directory entry initialization and parent link increment. Committing the directory entry before initializing the inode can result in a directory entry pointing to a garbage inode; committing before incrementing the parent's link count can lead to dangling links.

**Example 2:** `unlink`. To show how SSU handles multiple links to files and operations involving deallocations, we now describe how `unlink` works. Figure 4 shows the dependencies between each step in this operation. As before, each node represents a durable object at each step in the operation, with inodes in dark gray, directory entries in white, and data pages in gray. Figures 7 (inodes), Figure 9 (directory entries), and Figure 10 (data pages) show the full dependency relationships for each of these structures.

The `unlink` operation is initially passed a volatile directory entry structure maintained by the **Virtual File System** (**VFS**) layer. We use this to look up the durable inode and directory entry to unlink, both of which are initially in the `Start` typestate. First, we clear the directory entry's inode, which makes it invalid for future lookups. The inode's link count is now greater than the number of directory entries pointing to it, but this cannot cause dangling links in the event of a crash and is thus safe. We can now durably decrement the inode's link count, which requires passing in an immutable reference to the directory entry (shown in Figure 4 with a dotted arrow) to ensure the link count is not decremented before a linked directory entry has been cleared. The directory entry can now be deallocated. If the freed directory entry was the last live entry in a page, that page may now be safely deallocated. We omit these steps for brevity, but they are similar to those taken to deallocate data pages later in this example.

The next step depends on whether the inode's link count is now zero. This is checked at runtime by a function that returns the inode in the `Complete` state if the link count is greater than zero, and returns both the inode in `UnmapPages` state and a list of pages in `ToUnmap` state if the link count is zero. If the inode's link count is greater than zero, there is still at least one directory entry pointing to it, so the inode and associated pages should not be freed and the operation is complete. If the link count is zero, there are no more links to the inode, so the inode and its pages can be deallocated. SquirrelFS uses a backpointer-based page management approach (Section 3.4) in which each page points to the inode that owns it, so the pages must be deallocated before the inode to prevent dangling pointers in the event of a crash. If, instead, the inode pointed to a structure containing the locations of its pages, the inode would need to be freed first.

### 3.4 Implementation

We implemented SquirrelFS in Rust with 7500 LOC. It uses bindings from the Rust for Linux project [23] to connect to the Linux VFS layer. Figure 5 shows SquirrelFS's architecture. We also built a model of SquirrelFS in the model-checking language Alloy [32] to check its design for crash consistency issues. We describe our experience developing SquirrelFS in Section 4.

**Overview**. The design of SquirrelFS combines aspects of FreeBSD's FFS [47] and PM file systems such as NOVA [65] and WineFS [34]. Like FFS, it has a simple on-storage layout, and uses soft updates. Like other PM file systems, SquirrelFS uses volatile index structures that are built when the file system is mounted.

SquirrelFS's design was primarily influenced by two factors. First, we wanted to keep dependencies as simple as possible and avoid nested persistent structures that are difficult to represent in

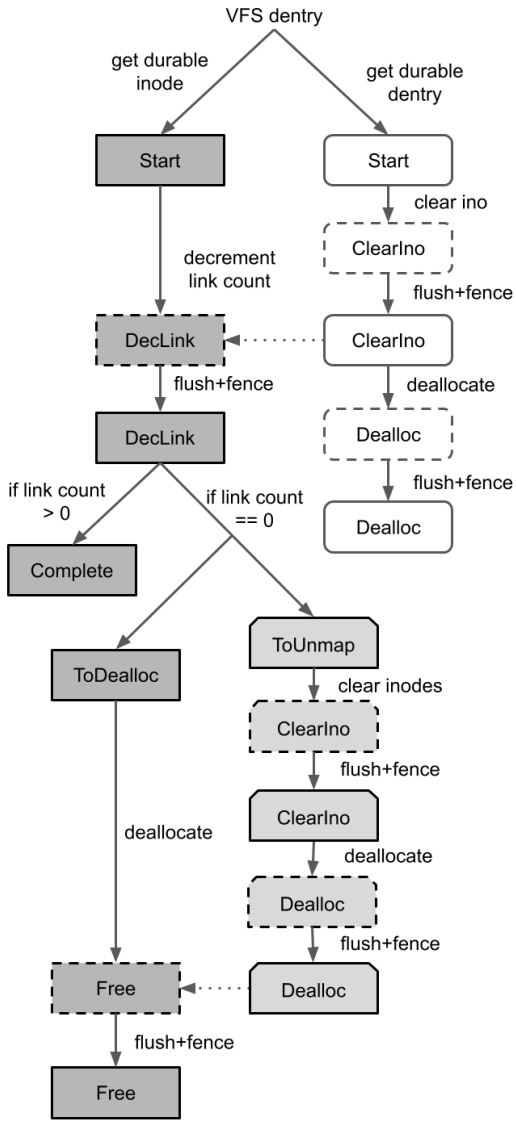


Fig. 4. The figure shows the persistent updates and corresponding dependencies made during `unlink`. Inodes are dark gray rectangles, directory entries are white rounded rectangles, and page lists are light gray pentagons. Each object is labeled with its operational typestate and its outline indicates whether it is clean (solid) or dirty (dotted). A dotted arrow indicates that the typestate transition requires a reference to an object in a particular typestate but does not modify that object's typestate.

typestate. Second, we assume the x86 PM persistence model in which only aligned updates of 8 bytes (or smaller) are crash atomic. We also assume volatile CPU caches. In this model, persistent addresses can be accessed via regular memory stores, but the corresponding cache line must be flushed or written back before updates become persistent. A memory barrier like a store fence must also be invoked to correctly order stores [56]. Durable structures may also be updated via cache-bypassing non-temporal store instructions, which still require a store fence for persistence

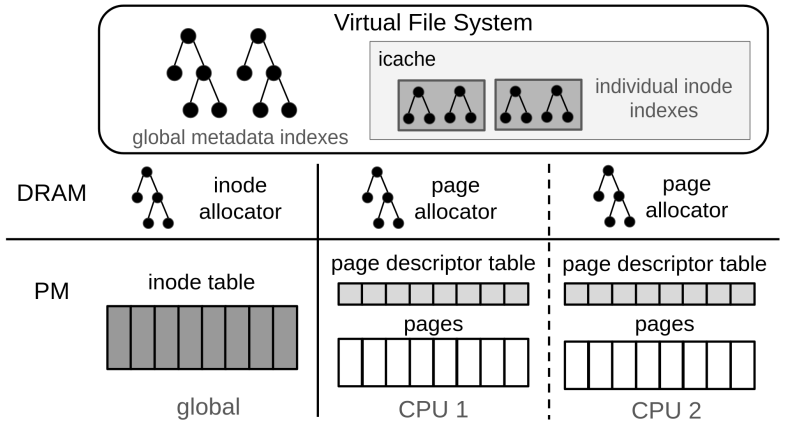


Fig. 5. The figure shows the main components of SquirrelFS. Each CPU has its own pool of pages and private page allocator. The inode allocator is shared between all CPUs. Volatile indexes are stored in VFS data structures.

ordering. This programming model influences the structure of persistent objects and restricts the set of legal orderings. SquirrelFS also supports systems with durable CPU caches (e.g., with eADR), which eliminate the need for explicit cache line write backs. Typestate checking is also useful for these systems, as store fences are still required to order memory stores and the high-level update dependencies are the same as with volatile CPU caches.

All system calls in SquirrelFS are synchronous, meaning that updates to durable structures made by each system call are durable by the time the system call returns. As such, `fsync` is a no-op in SquirrelFS. Metadata-related operations are also crash-atomic. Data-related operations are not atomic in the current SquirrelFS prototype, which matches the default behavior of other PM file systems like NOVA [65]. These operations could be made atomic by using copy-on-write to update file contents.

**Persistent layout**. SquirrelFS uses a simple layout to reduce the complexity of update dependencies. At mount time, the entire PM device is mapped into SquirrelFS's address space using DAX (direct access) management functions provided by the kernel [43]. SquirrelFS divides this mapped region into four sections: the superblock, the inode table, the page descriptor table, and the data pages. The inode table is an array of all of the inodes in the system. SquirrelFS reserves enough space for approximately one inode for every 32KB of data (eight pages).

The page descriptor array contains page metadata. Rather than having inodes point to the pages they own, each page descriptor contains a backpointer to its owner (similar to NoFS [13]) and stores its own metadata (e.g., its offset in the file). This approach simplifies dependency rules for updates involving page allocation and deallocation. All remaining space after the page descriptor table is used for data and/or directory pages.

All PM allocations are managed internally by SquirrelFS. As we discuss next, these allocators are kept only in volatile memory, so there is no allocator-related metadata stored on PM.

**Volatile structures**. SquirrelFS's persistent layout simplifies typestate and update dependency rules, but it is not amenable to fast lookups. Therefore, SquirrelFS uses indexes in DRAM to speed up lookup and read operations. Each inode in the VFS inode cache has a private index for the resources it owns; index data for uncached nodes is stored in the VFS superblock.

Like many other PM file systems, SquirrelFS uses volatile allocators: allocation information is not stored in a persistent manner, but rather rebuilt each time the file system is mounted. It uses a

per-CPU page allocator and a single shared inode allocator (which could be converted to a per-CPU allocator to improve scalability). The allocators use free lists backed by kernel RB-trees.

SquirrelFS's indexes and allocators are rebuilt by scanning the file system when SquirrelFS is mounted. Any persistent data structure that is zeroed out is considered free during the rebuild scan and is added to the corresponding free list. During the scan, SquirrelFS keeps track of allocated objects and uses them to traverse the file system tree and build the in-memory indexes. Data structures with any non-zero bytes are counted as allocated. After a crash, it is possible for an object to be allocated but invalid, e.g., a directory entry with a name but no inode pointer, or an inode that is not reachable from the root. These objects are zeroed out and added to the free lists during recovery. Thus, only updates that change an object's validity or update user-visible state in place need to be crash-atomic, since any invalid objects will be cleaned up after a crash.

**Synchronous Soft Updates.** SquirrelFS uses an implementation of SSU for crash consistency. As shown in Figure 3, operations that involve creation of new objects must first durably allocate and initialize resources before linking them into the file system (setting the directory entry's inode in the example) to enforce rule 1 (never point to a structure before it has been initialized). Deallocation proceeds in reverse; links are first cleared, then the object itself is deallocated by zeroing all of its bytes. SquirrelFS enforces rule 2 of soft updates (never re-use a resource before nullifying all previous pointers to it) by treating durable objects that are not completely zeroed out as allocated and by ensuring via typestate that pointers to the object are cleared before the object can be zeroed.

**Typestate transition functions.** SquirrelFS updates the typestate of objects via *typestate transition functions*. These functions take ownership of the original object, modify it, and return it to the caller with the new typestate. These functions are defined only on certain typestates to ensure they are called in a safe order. For example, the typestate transition function `commit_dentry()`, shown in Listing 3, is only defined for directory entries with type `Dentry<Clean, Alloc>`, and also takes ownership of an inode of type `Inode<Clean, Init>`. Calling `commit_dentry()` out of order – e.g., on a directory entry that has not yet been persistently allocated – is a potential crash-consistency bug and results in a compiler error.

**Concurrency.** SquirrelFS supports concurrent file-system operations. It relies on VFS-level locking on durable resources like inodes. This locking, together with Rust's type system, ensures that each resource has only one owner – and only one type – at any time, enabling strong typestate-based compile-time checking. SquirrelFS uses internal locks to protect its allocators and indexes.

**Building a model with Alloy**. While the typestate pattern can enforce a given operation order, it cannot verify that this order is crash consistent. To gain more confidence that SquirrelFS's design is crash consistent, we built a model of SquirrelFS in the Alloy model checking language [32].

Alloy provides a language for specifying transition systems and a model checker to explore possible sequences of states (traces) of these systems. Alloy's implementation is based on a logic of relations; each system is composed of a set of constraints that define a set of structures and the relations between them, and the model checker uses constraint solving to find traces.

In SquirrelFS, there is roughly a one-to-one mapping between typestate transitions in the Rust implementation and the next-state predicates in the Alloy model. Each next-state predicate specifies the states in which the transition may occur and the changes it makes to the model's state. The model includes next-state predicates for typestate transitions and persistent updates. It also includes transitions that model crashes and recovery, which let us check SquirrelFS's design for crash-consistency bugs.

Each persistent structure in SquirrelFS is represented by a corresponding structure, also called a signature, in Alloy. The model also includes a `Volatile` signature that is used to model volatile

```
pred inc_link_count_mkdir [i: DirInode, op: Mkdir] {
    // *Guards* specify when this transition can occur
    initialized[i] and op in i.i_rwsem_excl and
    lt[i.link_count, max] and isFalse[Volatile.recovering]

    // *Frame conditions* specify what parts of system
    // state are unchanged by the transition
    inode_values_unchanged_except_lc
    pointers_unchanged
    dentry_names_unchanged
    (all i0: Inode - i | unchanged[i0.link_count]
        and unchanged[i0.prev_link_count])
    (all o: PMObj - i | unchanged[o.typestate])
    locks_unchanged
    ops_unchanged_except_mkdir[op]
    unchanged[op.inode_typestate]
    unchanged[op.dentry_typestate]
    page_values_unchanged
    Volatile.recovering' = Volatile.recovering
    Volatile.allocated' = Volatile.allocated

    // *Effects* specify how the transition changes system state
    make_dirty[i]
    i.link_count' = add[i.link_count, 1]
    i.prev_link_count' = i.prev_link_count + i.link_count
    i.typestate' = IncLink
    op.parent_inode_typestate' = IncLink
}
```

Listing 4. A transition predicate in Alloy that increments a directory inode's link count during a `mkdir` operation.

aspects of the file system like its indexes. Each typestate is represented by a signature, and instances of persistent structures are mapped to their current typestate. Each file system operation is also represented by a signature, and relations map system calls to instances of persistent objects they are operating on as well as other volatile state (e.g., the locks held by that operation). We use this to model concurrent file-system operations.

**Alloy model example.** Listing 4 contains the source code of a transition that increments an inode's link count during a `mkdir` operation. Transitions in Alloy models are defined as predicates that are true in steps where the transition is applied and false at all other times. The predicate is evaluated based on the conjunction of each line in its body. The next value of a variable is referenced with `'` syntax; for example, `i.typestate' = IncLink` is true if `i`'s typestate in the next step of the trace (Alloy uses single-equals for equality).

This predicate takes two arguments: the inode to modify and a representation of the `mkdir` operation. Transition predicates Alloy models are generally organized into guards, frame conditions, and effects. The guards specify what must be true in a given state for the transition to occur. In

this predicate, the operation must hold the inode's lock, the inode's link count must be less than the maximum, the system must not be recovering from a crash, and an `initialized` predicate must hold. Incrementing an inode's link count is always safe in soft updates, so this operation has relatively weak guards. A transition's guards describe the typestate dependencies of the operation as well as aspects like which locks that are held during the operation and runtime error handling.

The frame conditions describe what parts of the system state may not change in this transition and must include all mutable parts of system state that should not be modified. Frame conditions do not directly correspond to any part of typestate transitions written in Rust, but are necessary in Alloy as the model checker is free to arbitrarily change aspects that are not captured by the frame conditions. Our model contains a set of macros and predicates to make frame conditions more concise; for example, `locks_unchanged` specifies that no locks are acquired or released in this transition.

The effects specify what parts of the system state do change during this transition. Some effects correspond directly with a durable or typestate update in the implementation of the corresponding transition function in SquirrelFS; for example, line 22 of Listing 4 specifies that the inode's persistence typestate becomes `Dirty`, and line 23 specifies that its link count is incremented. The model also tracks previous link count values between the last flush/fence and this update so that it can roll back to previous values in the event of a crash. Line 24 specifies that the current link count is added to the set of previous link counts for this inode. The model also tracks the current typestate of this inode expected by the `mkdir` operation (line 26), which helps maintain the association between the `op` argument and the persistent objects it modifies across multiple transitions.

### 3.5 Typestates

Table 2 describes the three persistence and 22 operational typestates used in SquirrelFS to represent Synchronous Soft Updates dependencies. SquirrelFS additionally uses traits to define sets of typestates for which a single operation can safely be performed on an object with any of the included typestates. For example, the `AddLink` trait is implemented by `Alloc` and `IncLink`, two inode typestates in which it is safe to add a link from a new directory entry to that inode.

**Recovery typestates.** Several typestates are used only during recovery: `Recovery`, `TooManyLinks`, and `Recovering`. Most durable operations during crash recovery can use existing functionality and typestates, as most of these updates involve deallocating orphaned structures that are not visible from the file system root.

`Recovery` is a special typestate used when reading a persistent object after a crash. Functions that return an object with typestate `Recovery` do not perform the same validity checks as functions that return an object with typestate `Start` and can thus be used in scenarios where, e.g., an object was incompletely initialized before a crash.

`TooManyLinks` is used to represent an inode whose link count is higher than its actual number of links. Like traditional soft updates systems, SquirrelFS allows an inode's link count to be incremented even if doing so may cause a link count leak in the event of a crash. Unlike other systems, SquirrelFS detects such leaks and fixes them during crash recovery by counting the number of directory entries that refer to each inode and comparing the count to the inode's real link count. The only legal operation on a `TooManyLinks` inode is a special `recovery_dec_link` function that durably sets the inode's link count to the correct value.

`Recovering` is used only to represent a source directory entry in a rename operation that is interrupted by a crash in Step ❷ or ❹ in Figure 2. In both steps, the destination directory entry's rename pointer must be cleared, either to roll the operation back (Step ❷) or forward (Step ❹), but the source directory does not need to be modified yet. Rename pointer fixes run before orphaned

Table 2. The Table Lists all Typestates used by SquirrelFS

| Type | Associated structures | State | Meaning |
|---|---|---|---|
| Persistence | All | `Dirty` | One or more updates to this object have not been flushed from CPU caches. |
| | | `InFlight` | All outstanding updates to this object have been flushed but not followed by a store fence. |
| | | `Clean` | All outstanding updates to this object have been flushed and fenced. |
| Operational | All | `Start` | Object has been read from PM in an initialized state. |
| | | `Free` | Object has been freed, or read from PM in an uninitialized state. |
| | | `Alloc` | Object has just been allocated. |
| | | `Recovery` | Indicates that an object has been read during recovery and may be in an invalid state. |
| | Inodes | `IncLink` | Object's link count has been incremented. |
| | | `DecLink` | Object's link count has been decremented. |
| | | `IncSize` | Inode's size has been increased. |
| | | `DecSize` | Inode's size has been decreased. |
| | | `TooManyLinks` | Indicates that an inode read during recovery has an incorrect link count and needs to be repaired. |
| | | `UnmapPages` | Indicates that an inode's pages can be unmapped safely; once they are, the inode can be deallocated. |
| | Page desc. | `Zeroed` | All bytes in the page have been zeroed. |
| | | `Writable` | Page's backpointer is set; page can now be written to. |
| | | `Written` | Page has had data written to it |
| | | `ToUnmap` | Indicates that a page descriptors's inode number field can be safely cleared. |
| | Dentries | `SetRenamePointer` | Destination dentry's rename pointer has been set to point to source dentry. Only used with destination directory entry. |
| | | `InitRenamePointer` | Destination dentry's inode number has been set to target inode. Only used with destination directory entry. |
| | | `Renaming` | Rename pointer to source dentry has been set. Only used with source directory entry. |
| | | `Renamed` | Destination dentry's inode number has been set to target inode. Only used with source directory entry. |
| | | `Recovering` | Represents a directory entry whose inode number does not need to be modified during post-crash rename cleanup. Only used with source directory entry. |
| | Dentries & page desc. | `ClearIno` | Object's inode number field has been cleared. |
| | | `Dealloc` | Object has been persistently deallocated. |
| | Inodes & dentries | `Complete` | The current operation has finished updating this object. |

It categorizes each typestate as Persistence or Operational, lists the durable data structures it is used with, and describes its meaning.

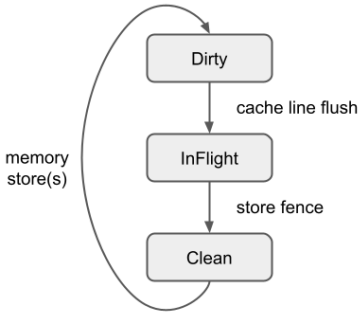


Fig. 6. Persistence state dependencies in SquirrelFS.

resource cleanup; if the source directory entry is orphaned after the rename is completed or rolled back, it will be deallocated later.

**Metadata updates.** Operations that modify file metadata like link count, size, or permissions need to update an existing inode in place. Each of these fields fits within 8 bytes and can be updated atomically. SquirrelFS does not currently support atomic updates to multiple separate file attributes, as the attributes may not all fit within a single atomic update. SSU does not provide a mechanism for arbitrary atomic updates (e.g., a journal), but this limitation could be overcome by using a technique like copy-on-write to perform non-atomic updates to a separate attribute data structure, then atomically updating an inode-resident pointer to that structure.

Some metadata operations, such as changing link count or file size, are either dependent on or are depended on by other operations. For example, as shown in Figure 3, the parent inode's link count must be durably incremented before setting the new directory entry's inode number in `mkdir` operations. These operations have special typestates used to enforce these dependencies. Others, such as setting permissions or changing file ownership, are completely independent of other durable operations and can be performed at any time. Updating these attributes is permitted in any typestate and does not change the inode's operational typestate, but does set its persistence typestate to `Dirty`.

**Typestate dependencies.** Figures 6, 7, 8, 9, and 10 illustrate the dependency relationships between typestates in SquirrelFS. Each typestate is represented by a gray box connected by solid arrows representing typestate transitions. The arrows are labeled with the operation or condition associated with that transition. Dotted arrows represent *implicit* typestate transitions. These are usually transitions from `Complete`, a typestate indicating that an operation has completed all updates to an object that is still in use, to `Start`, the starting state of initialized persistent objects in system call handlers. This transition is not associated with an explicit function invoked by SquirrelFS; rather, it occurs implicitly between when the typestate wrapper around a persistent reference is dropped and when the same persistent value is read again.

Most typestates are used with multiple types (see Table 2), but the dependencies and transition functions differ between types of persistent objects. For simplicity, these diagrams do not reflect all dependency relationships in SquirrelFS. They do not include recovery typestates and do not show full dependencies for operations that interact with multiple persistent objects. Typestate transitions that have a dependency on the operational typestate of another object that is not shown are marked with an asterisk. While these figures present persistence and operational typestate separately, they are used together in SquirrelFS to determine whether a transition is safe; for an example, see Figure 3. An object must be `Clean` before an operation that updates its operational

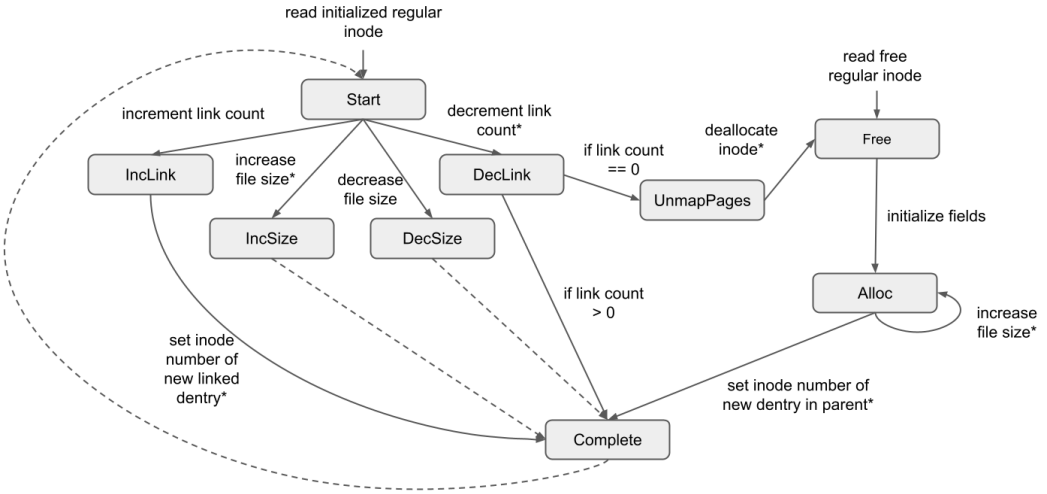


Fig. 7. Regular inode update dependencies in SquirrelFS.

typestate can be performed. However, independent transitions may change the typestate of some objects while others are `Dirty` or `InFlight`.

Figure 6 shows the dependencies between the three persistence typestates used in SquirrelFS. Persistence typestates are maintained separately from operational typestates and apply to all types of persistent objects. Each typestate wrapper type implements `flush` and `fence` functions to transition from `Dirty` to `InFlight` and `InFlight` to `Clean`, respectively. Any function that modifies a `Clean` persistent object acts as a transition from `Clean` to `Dirty`.

Figure 7 shows dependencies between typestates associated with regular inodes in SquirrelFS. Regular and directory inodes are distinguished using a type parameter and are thus generally treated as different types with some shared functionality, so they have separate typestate dependency diagrams. Increasing a regular inode's size also requires a page descriptor structure with a typestate indicating that we have already written some additional data to this file to ensure that a crash cannot accidentally expose garbage data. SquirrelFS maintains as an invariant that an inode's link count is always greater than or equal to the true number of links, so decreasing the inode's link count also requires a directory entry whose inode number has been cleared. The transitions from `IncLink` and `Alloc` to `Complete` do not make any durable modifications to this inode; rather, they are made by a function that updates a directory entry's inode number. The transition to `Complete`, which has no explicit transition functions to other states, ensures that an `IncLink` or `Alloc` inode can only be used with a single directory entry update.

Figure 8 shows dependencies between typestates associated with directory inodes. Directory inode dependencies are similar to those of regular inodes except that the conditions for transitions associated with link counts are slightly different and directory inodes do not use the `{Inc,Dec}Link` typestates. The specific directory entry operation associated with the transitions from `IncLink` to `Complete` is also different, since a directory inode's link count is increased only when a new child directory entry is created.

Figure 9 shows dependencies between typestates associated with directory entries. Along with similar allocation/deallocation transitions to inodes, SquirrelFS uses several directory-entry-only typestates in rename operations to provide a fully crash-atomic implementation of the `rename`

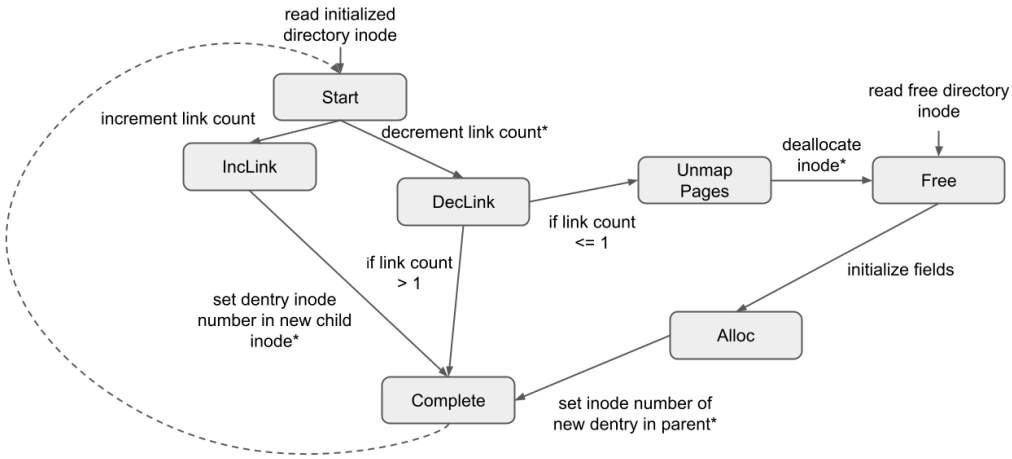


Fig. 8. Directory inode update dependencies in SquirrelFS.

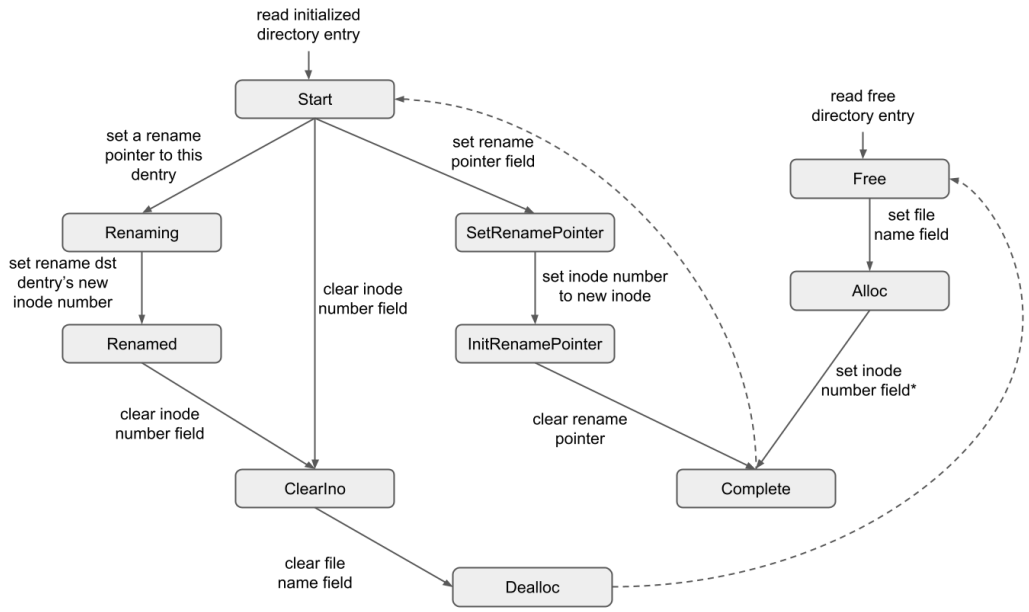


Fig. 9. Directory entry update dependencies in SquirrelFS.

system call. `Renaming` and `Renamed` represent the state of the source directory entry after the rename pointer has been set and after the destination directory entry's inode number has been updated, respectively (steps ❷ and ❸ in Figure 2). `SetRenamePointer` and `InitRenamePointer` represent the state of the destination directory entry as its rename pointer and new inode number, respectively, are set. The same functions simultaneously update the typestates of both directory entries involved in the `rename` operation in both cases.

Figure 10 shows dependencies between typestates associated with page descriptors and page contents. For simplicity, SquirrelFS uses a single wrapper structure to represent modifications

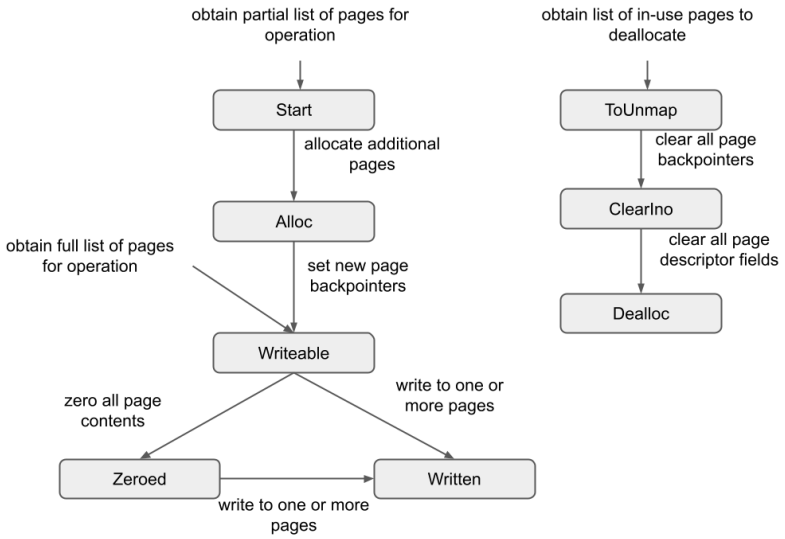


Fig. 10. Page update dependencies in SquirrelFS.

to both page descriptors and the contents of their associated pages. The typestate of these structures describes the state of a list of logically-sequential pages in a file, and typestate transitions on these structures update the state of all pages in the list. SquirrelFS tracks the typestate of collections of pages, rather than per-page typestate as with other types of persistent objects, due to language limitations and the inherent challenge of tracking typestate for an arbitrary number of objects. For more detail, see Sections 4.3 and 6.2. This diagram is not cyclic because the lists used in each operation are constructed on-demand with the pages required for that operation, so there is no implicit transition from, e.g., `Complete` to `Start` as there is for other structures.

SquirrelFS may obtain an incomplete list of pages if the required pages have not already been allocated, e.g., during an append operation. The caller passes the wrapper constructor the range of desired pages; if all pages in that range exist, the constructor returns a list in the `Writeable` state. Otherwise, it returns a list in the `Start` state, which cannot be written to or zeroed out until the missing pages have been allocated.

**Code example.** Listing 5 shows a typestate transition function used by SquirrelFS as part of `mkdir` operations. The shown code is a slightly simplified version of the real function from SquirrelFS. This function completes a `mkdir` operation by setting the new directory entry's inode number to that of a newly-allocated directory inode. The function is implemented only for `DentryWrapper` objects with type parameters `Clean` and `Alloc`.

This function (and any other functions defined in this `impl` block) can only be called on `DentryWrapper` objects with the specified typestates. This function also takes, and updates the typestate of, two `Clean` directory inodes: one in the `Alloc` state and one in the `IncLink` state. Together with the `self` parameter, these three parameters specify the typestate dependencies for this operation and ensure that the new directory entry cannot point to the new inode until the new inode is fully allocated and its parent's link count has been incremented.

Note that although `DentryWrapper` takes two type parameters, it does not store values of these types at runtime. However, the Rust compiler requires that a `struct` that is generic over

```
impl DentryWrapper<Clean, Alloc> {
    pub fn set_dir_ino(
        self,
        new_inode: InodeWrapper<Clean, Alloc, DirInode>,
        parent_inode: InodeWrapper<Clean, IncLink, DirInode>,
    ) -> (
        DentryWrapper<Dirty, Complete>,
        InodeWrapper<Clean, Complete, DirInode>,
        InodeWrapper<Clean, Complete, DirInode>,
    ) {
        self.dentry.ino = new_inode.get_ino();
        (
            DentryWrapper {
                state: PhantomData::<Dirty>,
                op: PhantomData::<Complete>,
                dentry: self.dentry,
            },
            InodeWrapper::new(new_inode),
            InodeWrapper::new(parent_inode),
        )
    }
}
```

Listing 5. A typestate transition function used by `mkdir`. Typestates are shown in bold.

a type `T` own a value of that type. The `PhantomData` type seen on lines 13 and 14 is a Rust standard library type that can be used to satisfy this requirement without using unnecessary space. A `struct` with a field of type `PhantomData<T>` appears to own a value of type `T` to the compiler, allowing this check to pass. However, `PhantomData<T>` is a *zero-sized* type, meaning that the field takes up no space at runtime. SquirrelFS types with typestate use `PhantomData` to meet the compiler requirement without using any runtime resources for typestate. The type argument to `PhantomData` can usually be inferred automatically, but we include it here for clarity.

### 3.6 Limitations of the Approach

It is important to note that the typestate-based approach used in SquirrelFS is not as powerful as full verification. Fully-verified systems, such as the FSCQ file system [11], use theorem provers that can prove a wide variety of complex properties. For example, a developer could prove, if required, that the system only uses even-numbered inodes for files.

In contrast, our typestate-based approach can only check *ordering-based* invariants. Our approach could be used to verify that functions are called in a specific order; for example, our approach can ensure that a file is not linked into the file-system tree before it is allocated. However, it does not verify the implementation of each function that is called.

Thus, full verification is significantly more powerful and general, but it pays a cost in terms of complexity and development time. Our approach is more targeted and ordering-based, but allows quick feedback and incremental development.

We believe this approach is a valuable addition to the repertoire of tools we have for building correct file systems. This approach should be used alongside runtime testing and model-checking approaches.

### 3.7 Relevance beyond PM

While we have designed SquirrelFS for persistent memory, SquirrelFS would be relevant for any storage technology with byte-addressability and low latency. The Compute Express Link [14] standard will support attached memory devices, including PM, via the Type 3 (CXL.mem) protocol. These CXL-attached PM devices will have the same interface and persistence semantics as current NVDIMMs, though performance will be lower [4].

SquirrelFS, and SSU file systems in general, could be used on CXL-attached memory. As SquirrelFS's mount performance and memory footprint are tied to the size of the device, they may worsen with significantly larger-capacity devices. Further work will be required to optimize file systems based on our approach for such devices.

## 4 Experience Developing SquirrelFS

We now describe our experience with designing, developing, and testing SquirrelFS. We also discuss the challenges we faced during this process.

### 4.1 Development Process

**Designing SquirrelFS.** Our initial design closely followed that of BSD FFS [46], but most aspects eventually diverged due to differences between storage hardware and typestate considerations. We found that some data structures and crash-consistency properties were better suited for use with the typestate pattern than others. For example, we chose SquirrelFS's backpointer-based page management approach because it simplifies update dependency rules when allocating or deallocating pages. With backpointers, these operations involve a constant number of persistent updates and involve no additional durable structures. In contrast, tree or log-based approaches need extra persistent metadata and may require additional updates to balance the tree or free log space, both of which complicate dependencies and typestate management.

An important design decision we had to make was how granular typestate would be. One option was to use specific typestates to represent each fine-grained operation; e.g., have one typestate for initializing an inode's link count, another for setting its flags, etc. Another was to make each typestate more general, with transition functions potentially performing multiple persistent updates. More general typestates may sacrifice some bug-finding power, but they make the system easier to understand and develop. In SquirrelFS, we aimed at striking a balance by representing only operations that require a specific ordering with typestate. For example, when initializing an inode in SquirrelFS, the order in which the values of most fields are set is not relevant to crash consistency, as the contents of the inode are not visible until it is linked into the file system tree. Therefore, SquirrelFS uses only a single typestate (`Init`) to represent inode initialization, and another (`Committed`) to indicate when it has been added to the tree.

**Parallel model and system implementation.** We developed the Alloy model alongside SquirrelFS. This created a useful feedback loop in which the model supported the Rust implementation, and questions or changes to the implementation could be quickly reflected and checked in the model. We used an incremental development process, incorporating feedback from the Rust compiler and the model immediately as we implemented the system. Many transitions in the model could be translated directly into Rust typestate transitions, making the model a valuable guide for

implementing file system operations. When we made mistakes translating the model into Rust, typestate checking quickly caught these issues.

Alloy also includes a graphical user interface for visualizing traces of operations on the model. This was useful for both investigating invariant violations and seeing the set of transitions that occur in a given file system operation, which could be translated directly into system call handler implementations. It also demonstrated locations where multiple updates could safely share a single store fence, which helped us avoid redundant fences.

### 4.2 Finding Bugs

While developing SquirrelFS, we used a combination of typestate checking, model checking in Alloy, and dynamic testing to find bugs.

**Typestate checking.** Typestate checking in the implementation was successful at quickly catching both missing persistence primitives and higher-level ordering bugs; we provide an example of each.

- *Missing persistence primitives.* Our initial implementation of `write` was missing flush and fence calls after setting the backpointer of a newly-allocated page. This bug was immediately highlighted as an error by the compiler. Had this bug made it into the implementation, a crash could cause a file to have a size larger than the number of pages associated with it, causing errors when trying to read the file.
- *Incorrect ordering*. Our initial `rename` implementation mistakenly decremented an inode's link count before clearing the corresponding directory entry. A crash could result in a link count that is lower than the true number of links, leading to a dangling link if the inode is subsequently deleted.

Although we did not specifically check execution paths without crashes, the crash-consistency invariants encoded in typestate were general enough to detect some bugs in this code. For example, the compiler caught a bug where pages were not fully deallocated during `unlink`, which did not require a crash to manifest. Typestate-related compiler errors were relatively uncommon overall, since using the model as a guide for implementation helped us get ordering right early. However, it provided a crucial safety net to prevent subtle bugs when we did make mistakes.

**Model checking with Alloy.** The Alloy model found several high-level issues in SquirrelFS's design that would have otherwise been difficult to detect and time-consuming to fix, including the following examples.

- We initially believed that crash recovery would not be needed other than to fix space leaks. Alloy found an instance of the model where a crash during rename followed by deallocation of the destination directory entry could cause an invalid directory entry to reappear. Fixing this required the addition of recovery transitions.
- Early designs for SquirrelFS stored . and .. directory entries durably. We discovered via model checking that our original dependency rules for handling these directory entries during more complicated operations like `rename` were not correct. Ultimately, we decided to not store these entries, since they can be constructed at runtime using indexed information.

**Testing.** Neither the typestate pattern nor the Alloy model eliminated the need to test SquirrelFS. Our primary goal was to check crash-consistency, and we did not check any invariants that only impact regular, non-crash execution. We used handwritten tests and the xfstests suite [63] to test these unchecked parts of the code.

All bugs found through testing were in parts of SquirrelFS that were not checked by typestate or directly modeled in Alloy. Most bugs were related to updating volatile indexes or VFS inodes, e.g., failing to remove a deallocated object from an index or setting the wrong value in the VFS inode.

There were also bugs in the implementations of typestate transitions, which are not themselves verified; for example, the transition that wrote new file data to a page did not always calculate the offset for non-aligned writes correctly. Implementing bug fixes was quick since we did not need to modify the typestate-restricted interface to objects and there were no proofs to update.

### 4.3 Challenges Encountered

**Challenges with typestate.** It is easier to write typestate-checked code than it is to write verified code, but this comes at the cost of less powerful compile-time checking. For example, checking universally-quantified formulas (e.g., all pages in a file are allocated) is undecidable, and unlike verification-aware languages, the Rust compiler has no heuristics to attempt to solve them. As a result, we cannot ensure invariants such as "all objects in a set are in a certain typestate"; specifically, we can't encode this in typestate because the number of objects in the set is not known at compile time.

This became a problem when implementing file-system operations like `unlink`, where we would like to e.g., check that the backpointers of all pages belonging to the file are cleared before deallocating the inode. Such a check ensures that the system always follows soft updates rule 2 (never re-use a resource before nullifying all previous pointers to it); by clearing all of the page backpointers before deleting the inode, we ensure that none remain when the inode is eventually reused. However, it is impossible to check this property on arbitrary sets of pages if each page has its own typestate. We experimented with several workarounds, including forcing `write` operations to update no more than one page at a time (which was prohibitively slow and did not solve the problem for `unlink`), and storing typestate in page structures at runtime and manually adding assertions (which also impacted performance and lost the benefit of static checking).

Ultimately, we decided to use a single piece of typestate to represent *ranges* of pages (e.g., all of the pages in a file or a contiguous subsection). As shown in Figure 10, each typestate transition performs an operation on a range of pages, which is obtained on-demand. This solution moves some logic into the typestate transition functions, where it is not checked for crash consistency issues. This sacrifices some static checking power, but provides a simple interface for modifying and reasoning about collections of pages. For example, the transition from `Writeable` to `Zeroed` in Figure 10 iterates over each page in the list and zeroes it out before returning. If there is a bug in this implementation that causes it to skip some pages, it may be caught by standard testing but will not be identified by typestate checking. In practice, we found that operations on page ranges were longer and slightly more complex than other typestate transition functions, but they are not difficult to understand or test.

**Challenges with Alloy.** As SquirrelFS grew in complexity, it became harder to maintain the model and get useful feedback quickly. The model checker uses a SAT solver to check invariants, and the formulas representing a large model can take days or weeks to solve. We checked that traces with multiple concurrent operations were crash consistent, which increased the size of the problem further. To get faster feedback, we built a custom utility to run multiple independent instances of the model checker in parallel and split larger predicates into smaller, more concrete sub-checks.

It could also be difficult to determine whether a reported failure was a false positive. A particular challenge was dealing with *frame conditions*, predicates that specify what should not change in a given transition. Alloy is free to arbitrarily change any state that the current transition does not explicitly mention, so frame conditions are crucial to constrain the model to real traces. This behavior helps Alloy find corner-case bugs, but it also leads to false positives. To overcome this challenge, we built a syntax-based checker that parses the model using Alloy's API and checks

that each transition explicitly mentions all mutable structures in the model. The current version of the checker cannot detect all issues, but it detected many missing conditions that would have otherwise taken hours to catch via model checking.

### 4.4 Typestate beyond SquirrelFS

**Costs and benefits of typestate.** We do not have equivalent verified or unverified systems to compare with SquirrelFS in terms of development and debugging effort; however, in the authors' experience, designing and implementing SquirrelFS required more effort than a typical unverified system, but far less effort than a verified storage system.[1] We believe that debugging SquirrelFS was faster and easier than debugging an equivalent unverified system, as following the typestate-enforced ordering rules made it easier to implement the system correctly in the first place and reduced the number of bugs overall.

Using the typestate pattern for crash consistency represents a useful new point in the tradeoff space between runtime testing and full verification. While it comes at the cost of additional development effort compared to unverified systems to determine correct ordering rules and does not gain the same correctness guarantees as verified systems, it does eliminate an entire class of crash consistency bugs that are otherwise difficult to find and fix [36, 42, 49]. Furthermore, as the pattern builds ordering rules directly into a system's implementation, the rules will stay up to date and continue to prevent crash-consistency bugs as the system is developed further [29, 52].

**Broader applicability.** As the typestate pattern is a general approach for statically checking the order of updates to data structures, it is useful in a broad variety of contexts, several of which are described below.

- Volatile data structures: SquirrelFS does not use typestate to manage updates to volatile data structures, but prior work on typestate verification has focused entirely on such use cases [1, 58].
- Other types of storage systems: The typestate pattern could be used to enforce ordering invariants on durable updates in other types of storage systems (e.g., key-value stores) with different crash-consistency mechanisms. We note that crash-consistency mechanisms like journaling and copy-on-write do not achieve consistency entirely through ordering and would require auxiliary techniques to check properties like atomicity.
- Durable layout: SquirrelFS's on-storage layout is tailored to reduce the number of durable updates per file-system operation and to simplify ordering rules. Other layouts could also be used in typestate-checked storage systems, although the complexity of the ordering rules would increase.
- Asynchrony: The typestate pattern is compatible with asynchronous systems, although the ordering rules to enforce are much more complicated in such systems, as updates from different operations may be interleaved.

## 5 Evaluation

We seek to answer the following questions in our evaluation of SquirrelFS:

(1) What is the latency of different file-system operations on SquirrelFS? (Section 5.2)
(2) How does SquirrelFS perform on macrobenchmarks? (Section 5.3)
(3) How does SquirrelFS perform on real applications? (Section 5.4)
(4) How long does SquirrelFS take to mount and recover from crashes? (Section 5.5)

[1]For example, author LeBlanc recently worked on a durable log implemented in a verification-aware programming language, which took about 3 months of full-time work.

(5) What compilation, memory, and CPU overheads does SquirrelFS incur? (Section 5.6)
(6) Is SquirrelFS correct? (Section 5.7)
(7) How could SquirrelFS' performance be improved? (Section 5.8)

### 5.1 Experimental Setup

We evaluate SquirrelFS on a two-socket, 32-core machine with 128GB of memory and one 128GB Intel Optane DC Persistent Memory Module. The evaluation machine runs Debian Bookworm and Linux 6.3.

We compare SquirrelFS against ext4-DAX [43], NOVA [65], and WineFS [33]. We configure all three systems to provide metadata consistency but not data consistency to match SquirrelFS's guarantees. All reported results are the average of multiple trials. The red errors bars in Figure 11 indicate the minimum and maximum values recorded over all trials.

We cannot compare SquirrelFS to SoupFS [20], the only other soft updates PM file system, as it is not open source. We do not present a comparison of SquirrelFS against ArckFS [68], a PM file system that uses a soft-updates-inspired crash-consistency mechanism, because ArckFS runs entirely in userspace. Userspace file systems have significantly different performance characteristics from in-kernel systems due to the reduced kernel crossings and different resource management behaviors. Small tests on ArckFS indicate that while it has lower average latency than SquirrelFS on some system calls due to its lower software overhead, it must periodically perform costly in-kernel PM allocations to serve some system calls, incurring high tail latencies.

### 5.2 Microbenchmarks

We compare each system's latency by testing several file system operations: appending and reading 1KB and 16KB to a file, file creation, directory creation, renaming a directory, and unlinking a 16KB file. None of the tests call `fsync`.

The average latency over 10 trials of the tested operations are shown in Figure 11(a). The lowest latency file system in each test is either WineFS or SquirrelFS. Ext4-DAX has the highest latency on many operations because it interacts with the Linux kernel block layer for tasks like block allocation, which incurs additional software overhead. It achieves similar performance to the other systems on operations that do not go through the block layer (e.g., unlink). NOVA has higher latency on `mkdir` and `rename` than WineFS and SquirrelFS because operations that update multiple inodes require journaling in NOVA.

### 5.3 Macrobenchmarks

We evaluate SquirrelFS on the Filebench [60] storage benchmark suite. We run four workloads from the suite – fileserver, varmail, webserver, and webproxy – in their default configurations. Fileserver performs mostly writes with some whole file reads; varmail is half appends and half reads; webproxy appends to each file and reads from it several times; and webserver reads and occasionally appends to a log file. Figure 11(b) shows the average throughput in kops/sec for each file system on each workload. SquirrelFS achieves slightly better throughput than the next fastest system on fileserver and varmail (8% and 13% better, respectively) and within 10% of the fastest system on both webserver and webproxy. Fileserver and varmail perform many small appends, which SquirrelFS performs well on due to its lack of journaling. Webserver and webproxy are more read-heavy, which all systems perform roughly equally on. Ext4-DAX does not go through the block layer on reads and it benefits from data contiguity awareness, making its performance similar or better than the other systems on these workloads.

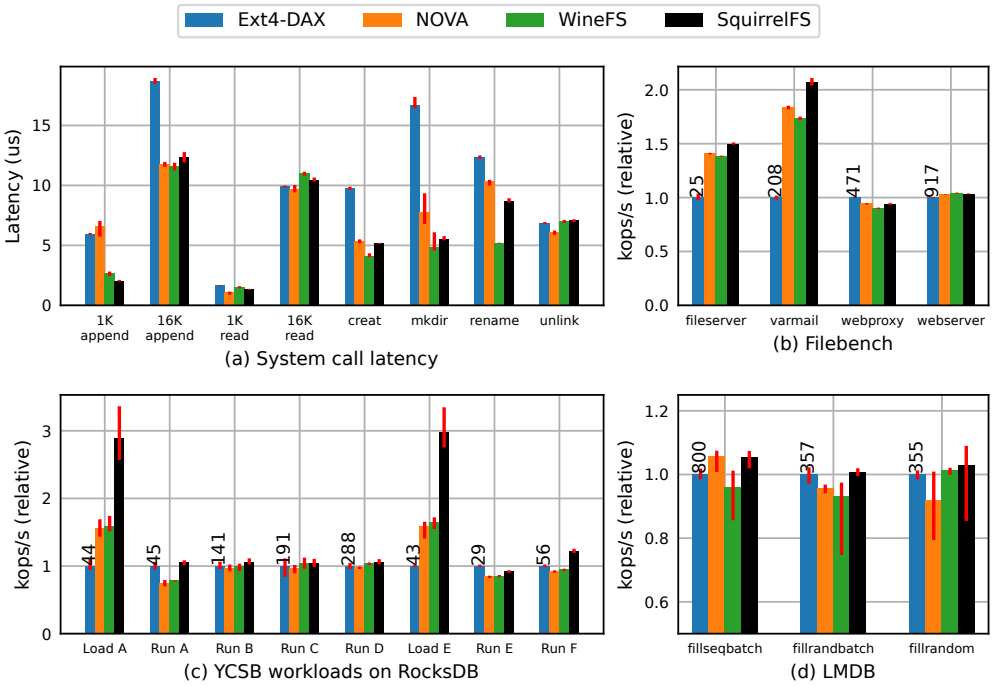


Fig. 11. This figure shows the performance of the evaluated file systems on different benchmarks and applications. (a) shows absolute latency of different file system operations; (b), (c), and (d) show the relative throughput in kops/s of each system relative to Ext4-DAX on filebench, YCSB on RocksDB, and LMDB respectively.

### 5.4 Applications

**YCSB on RocksDB.** We evaluate the four systems on RocksDB [55] using YCSB workloads [15]. We run all workloads on a 25GB database with 25M records, 25M operations, and 8 threads. All workloads are run using standard workload configurations and the default settings of YCSB, which uses system calls for all operations. Figure 11(c) shows throughput in kops/second relative to Ext4-DAX on each tested workload.

SquirrelFS outperforms the other systems on Loads A and E, which are 100% small inserts. As seen on the other benchmarks, SquirrelFS performs particularly well on small appends due to its lack of journaling or logging. Writes that require page allocation are particularly expensive in the other systems, as journaling/logging the new metadata incurs an additional 2-3us in NOVA and WineFS and 3-4us in Ext4-DAX. Ext4-DAX and NOVA both also journal or log metadata on every append, spending roughly 30% of each non-allocating call (approx 1-1.5us) managing journals/logs.

All file systems are within 10% of Ext4-DAX's throughput on Runs B, C, and D. All of these workloads are at least 95% small (4KB) reads, which all four systems achieve similar performance on.

SquirrelFS achieves the best throughput on Runs A and F, which are 50% reads and 50% updates (Run A) or read-modify-write operations (Run F). Ext4-DAX, NOVA, and WineFS all incur logging/journaling on these workloads; Ext4-DAX outperforms NOVA and WineFS because it has less journaling overhead for in-place updates and is more aware of data contiguity on reads.

Table 3. `git checkout` Performance

| | Checkout time (s) | | | |
|---|---|---|---|---|
| System | v3.0 | v4.0 | v5.0 | v6.0 |
| Ext4-DAX | 4.6 | 4.9 | 6.5 | 7.6 |
| NOVA | 4.6 | 4.9 | 6.5 | 7.2 |
| WineFS | 4.5 | 4.7 | 6.1 | 7.0 |
| SquirrelFS | 4.7 | 4.9 | 6.4 | 7.2 |

The table shows the time in seconds taken by each system to `git checkout` successive Linux versions. SquirrelFS achieves similar performance to state-of-the-art PM file systems.

Table 4. File System `mount` Performance

| | Normal | | | Recovery | |
|---|---|---|---|---|---|
| System | mkfs | Empty | Full | Empty | Full |
| Ext4-DAX | 0.33 | 0.01 | 0.01 | 0.01 | 0.01 |
| SquirrelFS | 5.80 | 5.51 | 30.50 | 5.76 | 55.50 |

The table shows how long it takes to mount Ext4-DAX and SquirrelFS on a 128GB PM device. Times in the "Normal" column come from mounting a cleanly-unmounted system, and those in the "Recovery" column are obtained by modifying the file system to run recovery when mounting a cleanly-unmounted instance.

Ext4-DAX achieves the best performance on Run E, which is 95% range scans and 5% inserts. Ext4-DAX's contiguity-awareness and better fragmentation-prevention mechanisms help it outperform the other systems on larger read operations.

**LMDB.** We also run LMDB [16], a memory-mapped database, using `db_bench`'s `fillseqbatch`, `fillrandbatch`, and `fillrand` workloads. Each experiment uses 100M keys on an empty file system. Figure 11(d) shows the throughput in kops/sec for each file system on each workload. Each file system has throughput with 12% of the other systems. Most updates are done to memory-mapped files, so differences in the performance of system calls and metadata management designs have a reduced impact.

**Git.** We also evaluate each system by using `git checkout` to switch between major Linux kernel versions from GitHub. We start at v2.6.12 (the oldest version of Linux available on GitHub) and successively check out versions 3.0, 4.0, 5.0, and 6.0. Table 3 shows the average time for each checkout over 10 iterations. The time to check out a given version in each file system is within 8% of the other systems.

### 5.5 Mount Time

Table 4 shows how long it takes to mount SquirrelFS, compared to Ext4-DAX as a baseline, on a 128GB PM device with different contents. We report the average of 10 mount operations. We measure the time each system takes to run `mkfs` (i.e., to create an empty file system instance), to mount an empty system, and to mount a completely full instance. To create a full system, we create 128 directories and create 32KB files in each directory in parallel until failure.

SquirrelFS takes longer than Ext4-DAX in all setups. During `mkfs`, it must zero out almost the entire device, since SquirrelFS interprets non-zeroed-out data structures as allocated, and construct the allocators. Mounting an empty system is similar, except that SquirrelFS scans the inode and page descriptor tables rather than zeroing them. Mounting a full system takes longer

Table 5. Compilation Times

| System | LOC | Compile time (s) |
|---|---|---|
| Ext4 | 45K | 38 |
| NOVA | 16K | 20 |
| WineFS | 9K | 13 |
| SquirrelFS | 7.5K | 10 |

The table shows the size of each evaluated file system in lines of code and how long each system takes to compile as a loadable kernel module. Ext4's line count includes interleaved DAX and non-DAX code.

Table 6. Memory usage

| | MiB used | |
|---|---|---|
| System | Empty | Full |
| Ext4-DAX | 1 | 336 |
| NOVA | 1 | 64 |
| WineFS | 3 | 57 |
| SquirrelFS | 1,104 | 3,220 |

The table shows the memory used in MiB by each evaluated system when both empty and full.

because SquirrelFS has to scan more data structures (e.g., it now must read each page of directory entries to determine the file system structure) and allocate space for each index.

Table 4 also reports the time taken by SquirrelFS and Ext4-DAX to perform recovery procedures on a cleanly-unmounted device in the "Recovery" column. We added a mount-time argument to each system that instructs it to run recovery-related code, regardless of whether a crash occurred.

SquirrelFS takes additional time to mount when running recovery even when no cleanup is needed. If SquirrelFS detects that it was not unmounted cleanly, it constructs additional structures to keep track of orphaned objects and the true link count of each inode. It fills in these structures during the regular rebuild scan and uses them to free orphans and correct link counts at the end of the mount process. SquirrelFS also checks each directory entry for non-null rename pointers and either rolls back or completes any interrupted renames. In this experiment, Ext4-DAX's mount time is unaffected by running recovery code.

SquirrelFS's mount time could be improved by parallelizing some of its rebuild and recovery logic. For example, the inode and page descriptor table scans are completely independent and could be done in parallel. The file system tree rebuild logic could also be distributed across multiple threads.

### 5.6 Resource Usage

**Compilation.** SquirrelFS takes approximately 10 seconds to compile on our test machine, including typestate checking. This compares well to fully-verified systems; FSCQ [11] takes about 11 hours to verify, and VeriBetrKV [28] takes 1.8 hours (10 minutes when parallelized).

SquirrelFS also compiles faster than the other tested systems on the test machine. Table 5 shows the size of each system in lines of code and how long it takes to compile. SquirrelFS's more complicated typechecking does not noticeably impact its compilation time.

**Memory.** Table 6 shows the amount of memory used in empty and full instances of each evaluated system. We measured memory usage by checking the amount of available memory reported by `/proc/meminfo` before and after mounting and filling up the file system. The reported numbers are the average of five measurements. SquirrelFS uses significantly more memory in both cases than the other evaluated systems.

In an empty system, most of this space is taken up by free lists. SquirrelFS uses free lists backed by red-black trees to allocate inodes and pages. Each inode and page number in the free list is 8 bytes, and each red-black tree node uses 24 additional bytes to store its color and child pointers. In this experiment, SquirrelFS has approximately 4 million inodes and 32.5 million data pages, which result in free lists that use about 1GB of memory when full.

In a full system, most memory is used by SquirrelFS' indexes. The indexes use red-black trees to map inodes to information about their pages and/or directory entries, which is also stored in a second level of red-black trees. The exact memory usage depends on the number of directories and regular files in the system, as directories take more space to index since each directory's pages and dentries are indexed in memory. The dentry and data pages indexes are the most expensive, each occupying about 1GB. In this experiment, all indexes remain in the VFS inode cache along with a lock around each index and some additional per-file metadata; altogether, this uses another 1GB of DRAM.

**CPU.** SquirrelFS does not start new threads in any of its operations. We leave the use of more threads for operations like freeing pages, running crash recovery, etc. to future work.

### 5.7 Correctness

**Model checking.** We check that a correctness invariant always holds in all traces of our Alloy model. We bound traces to include two operations (which may be concurrent), 10 persistent objects, and up to 30 steps. The invariant includes both sanity checks on the model as well as file system consistency checks. The sanity checks ensure, for example, that objects will never end up with conflicting typestates. The consistency checks ensure that (1) objects always have a legal link count, (2) there are no pointers to uninitialized objects, (3) freed objects do not contain pointers to other objects, and (4) there are no cycles of rename pointers and directory entries are pointed to by at most one rename pointer.

**Testing.** We test SquirrelFS using a set of handwritten tests and the xfstests [63] test suite. SquirrelFS currently passes all supported tests (67) from xfstests' `generic` test suite. The rest of the tests use system calls or arguments that are currently not supported by SquirrelFS.

**Crash consistency.** We used Chipmunk [42] to test SquirrelFS for crash-consistency bugs. We modified Chipmunk's test generators to remove several system calls that SquirrelFS does not currently support but otherwise ran its full suite of systematically-generated tests and fuzzed the system for approximately 24 hours. Chipmunk did not find any ordering-related crash-consistency bugs in SquirrelFS, providing evidence that typestate-checked SSU is an effective mechanism for preventing such bugs. Chipmunk did find four crash consistency bugs in unchecked parts of SquirrelFS code, three in its rebuilding of volatile data structures and one in the body of typestate transitions in which a cache line flush was issued to the wrong address. As these are not caused by incorrect update ordering, the typestate pattern did not catch them at compile time. We found that using the typestate pattern in SquirrelFS made locating and fixing these bugs faster and easier, as we could focus on the specific regions of code that are unchecked and are thus more likely to have bugs.

### 5.8 Limitations and Improvements

As SquirrelFS is a research artifact focused on correctness rather than performance, its current design has several performance limitations. It uses more memory than the other evaluated systems

and takes longer to mount. SquirrelFS uses more memory because it maintains an index of the entire file system in volatile memory. Its longer mount times stem from additional work during its scan of the entire system and the need to allocate space for indexes and allocators as they are rebuilt. Recovering from a crash involves scanning more durable structures and keeping track of additional system information, which further increases post-crash mount times. We now discuss several ways that SquirrelFS's mount performance and memory usage could be improved.

**Parallelizing mount.** SquirrelFS runs all mount-time device scans and volatile data structure construction sequentially, and its mount time could be improved by dividing this work across separate threads. Parallelizing mount procedures is a well-known technique in PM file systems [33, 65]. During remount, SquirrelFS first scans the inode and page descriptor tables to determine which entries are allocated and which inode each allocated page points to. Since SquirrelFS uses per-CPU page descriptor tables, each table could be scanned in parallel. SquirrelFS' inode table is currently managed globally, but could also be divided among multiple threads to scan in parallel during mount.

After determining which inodes and pages have been allocated, SquirrelFS scans each directory entry page to determine which durable structures are reachable from the root of the system and build the global index. It performs a breadth-first search over the file system tree, starting at the root inode, by reading the valid directory entries in each directory page associated with the current inode. This scan could be performed by multiple threads in parallel, although it would require more synchronization than the previous step on shared data structures like the search queue and indexes. Finally, we could parallelize the construction of the inode and page free lists, which currently use an inefficient implementation that determines which table entries are free by iterating over lists of allocated entries.

There are also several recovery-specific operations that could also be parallelized. For example, checking for interrupted `rename` operations requires an additional directory entry scan before constructing the global index, which could also be divided among multiple threads. SquirrelFS also durably frees all orphaned data structures after a crash; this could easily be parallelized but is unlikely to have a performance impact, since we expect there to be relatively few orphaned structures after a crash.

**Durable indexes and/or allocators.** SquirrelFS currently keeps all indexes and allocators in volatile memory. Along with increasing its memory footprint, this also hurts mount performance, since space for these structures must be allocated at mount time. SquirrelFS could improve in both dimensions by moving these structures (partially or entirely) to PM. Allocators could be straightforwardly stored on PM, e.g., using bitmaps as in the original soft updates implementations [46].

Reducing the memory footprint of SquirrelFS' indexes is more complex. We now consider two possible approaches. First, SquirrelFS could be modified to use a durable layout that is more amenable to lookups, eliminating the need for a global index. For example, SquirrelFS could use a simple direct/indirect block design to manage file contents as in prior soft updates implementations [46]. The dependencies for this approach are more complicated than SquirrelFS' current backpointer-based page management, so this would require nontrivial changes to its typestates as well as its durable layout.

Second, we could store separate durable index structures without otherwise changing the layout of SquirrelFS. For example, NOVA and WineFS checkpoint allocator structures during clean unmount so that they can be rebuilt without a full device scan [33, 65]. This improves regular-case mount performance, but it does not eliminate the need for an expensive post-crash device scan and would not reduce DRAM usage in SquirrelFS. Another option would be to store these structures in a dedicated region of PM to ensure updates become durable quickly without needing to modify

the layout of the rest of the system. However, it would be difficult to keep these structures in sync with the main file system tree in the event of a crash. One potential solution would be to maintain per-file indexes and store some durable metadata (e.g., setting a bit or storing a checksum of the index contents) that can be used to determine if we crashed while updating a given index. After a crash, we would rebuild only the indexes that may have been corrupted by the crash, which would require a shorter recovery scan. To simplify this scan, we would most likely also expand SquirrelFS' typestates and dependencies to include updates to these indexes.

**Improved in-memory data structures.** SquirrelFS' memory usage could also be improved by simply using more space-efficient data structures for volatile indexes. This would also improve mount times over the current implementation by reducing the amount of memory that needs to be allocated during mount. For example, SquirrelFS' allocators are currently implemented using the Linux kernel's built-in red-black tree library, but an in-memory bitmap or a list of free page ranges would be much more efficient. Its directory entry index uses full file names as keys, which could be hashed to save space, and the dentry metadata structures stored in this index could be further optimized. SquirrelFS also currently uses separate indexes to map directory inodes to their pages and child directory entries for simplicity, but these indexes could be combined to save additional space.

### 5.9 Summary

SquirrelFS provides comparable performance to other PM file systems, while providing strong guarantees about its crash consistency. Due to the innovative use of typestate checking, we were able to implement SSU and gain confidence in its correctness. SquirrelFS gains an advantage over other file systems in write-dominated workloads, since soft updates avoids writing to a log or to a second copy of the data. The design of SquirrelFS trades off good common-case performance for slightly longer mount times compared to other file systems; we believe this is acceptable since crashes are rare. SquirrelFS compiles at the same rate as other PM file systems, despite the strong type checking.

## 6 Related Work

### 6.1 Storage Systems

**Rust for PM**. SquirrelFS was inspired by Corundum [31]. Corundum builds data structures whose low-level properties are checked using Rust's type system. For example, Corundum ensures that there are no pointers to volatile memory stored in persistent memory, and that persistent state is only updated in transactions. It focuses on lower-level persistent memory programming errors and cannot prevent higher-level logical bugs. Corundum also requires *all* updates to PM to be in transactions, which is overly restrictive for many systems. In contrast to Corundum, SquirrelFS checks high-level file-system crash-consistency properties using type-checking without placing constraints on how the file system is used.

**Soft updates for PM**. Two PM file systems use soft updates for crash consistency: SoupFS [20] and ArckFS [68]. Unlike SquirrelFS, SoupFS is asynchronous and uses background threads to flush updates. It uses byte-addressable updates to eliminate cyclic dependencies. ArckFS is a user-space PM file system built on the Trio architecture that uses synchronous, soft-updates-esque updates for simple operations (e.g., creating a file) and undo journaling in more complicated cases. Unlike ArckFS, SquirrelFS uses only synchronous soft updates for its crash consistency; the novel way in which SquirrelFS implements atomic rename (without journaling or copy-on-write) further differentiates it from ArckFS. Both SoupFS and ArckFS are written in C, and do not use Rust's type system to check their crash consistency.

**Storage systems in Rust**. Bento [48] is a framework for building in-kernel file systems in Rust. The corresponding file system from the Bento project, BentoFS, was designed for block devices. Bento does not utilize the type system of Rust to check file-system properties.

ShardStore [6] is a Rust key-value store used in Amazon S3 that uses an asynchronous soft-updates-inspired crash-consistency mechanism. The rules for when something should be written to storage in ShardStore were checked with DepSynth [62], a tool for synthesizing soft updates dependency rules. Unlike ShardStore, SquirrelFS uses a synchronous version of soft updates, and provides higher-level primitives like atomic rename; ShardStore does not utilize the type system to perform higher-level checks.

### 6.2 Guarantees and limitations of Typestate Checking

Prior work [19, 22, 40, 58] has established the formal guarantees that can be obtained from typestate checking, fundamental limitations of the approach, and the computational complexity of statically analyzing typestates in a program. In this section, we discuss these results and apply them to SquirrelFS and the Rust-based typestate checking used in its implementation.

**Theoretical properties of typestate checking.** Static program verification is computationally complex. Many problems in static analysis are at least NP-complete, if not undecidable or uncomputable. Frameworks for developing fully-verified programs, like interactive proof assistants and verification-aware programming languages, generally rely on sophisticated theorem provers such as Z3 [18] or other solvers [61]. To handle complex queries, these tools use heuristics and/or require a high degree of developer interaction based on a significant amount of work over many years in the verification community.

The computational complexity of tracking typestate in a program depends on the structure of the target program and the specific property being verified. Even conceptually-similar properties, such as ensuring that a file is not read after it is closed and ensuring that it is not read before it is opened, are in different complexity classes (here, P and PSPACE-complete, respectively). The presence of pointers and aliasing in a program significantly complicates typestate analysis; in the worst case, it is undecidable in a program with recursive data structures. For more discussion and full proofs of these results, see [22].

**Typestate checking in SquirrelFS.** Although SquirrelFS uses typestate to check properties about data structures containing references to other structures (e.g., directory entries containing inode numbers), it only accesses a finite and statically-known number of typestate-ful objects in each operation. When a durable object is accessed, SquirrelFS checks its contents to determine its current typestate (generally `Start` or `Free`) and constructs a wrapper structure with the current typestate that contains a reference to the durable object itself. Typestate transition functions act on the type of this wrapper, not the type of the durable object itself, which hides pointers from typestate checks and reduces complexity.

**Comparison to typestate-oriented languages.** Several programming languages have been designed partially or entirely around typestate-based static analysis. We focus here on two such languages: Vault [19], a C-like language for low-level systems used to build Windows 2000-compatible device drivers, and Plaid [1, 59], a general-purpose language that extends standard object-oriented programming with dynamically-checked typestates. These languages include specific features for reasoning about typestates, such as the ability to maintain multiple mutable aliases to a single value or to maintain typestate information about an arbitrary number of objects in a collection. We present a comparison of these languages to Rust to demonstrate other capabilities of typestate-related techniques not discussed or used in SquirrelFS, and to provide intuition about the limitations of the approach. Rust's typestate support is less powerful than that of these other languages, so

properties that can be checked with typestate in a Rust program are a subset of those that could be checked in programs written in these typestate-oriented languages.

**Vault.** The Vault programming language is based on C and builds on the idea of typestate to support tracking states of aliased objects using prior theoretical frameworks [17]. Vault's key insight is that adding a level of indirection between an object's type and its compiler-tracked state allows for more flexible and powerful static checking. To do this, Vault tracks resources using a set of *keys*, which describe the current state of each object and can be checked in function pre- and post-conditions.

Unlike Rust, Vault allows multiple mutable aliases to individual objects, although developers are required to explicitly state which names alias to which objects. Rust does not support multiple mutable aliases to the same resource without runtime checking via the interior mutability pattern or use of unsafe Rust, both of which give up some strong guarantees from the compiler. We did not find the inability to check properties about multiple mutable aliases problematic in SquirrelFS, as dealing with aliasing restrictions is a standard part of Rust development, and we generally did not need multiple aliases to the same durable structures. Vault also provides a way to statically reason about the state of arbitrary numbers of objects in a collection by storing key information alongside the corresponding resource. In Rust, typestates are part of an object's type, and objects of different types cannot be stored in a collection and still benefit from strong compile-time checks. This imposed a limitation on SquirrelFS's design that all objects with typestate had to be used only in fixed, statically-known numbers, which weakened the guarantees we could obtain in some operations.

Vault's main case study was a floppy disk driver compatible with Windows 2000 that tracked low-level states associated with, e.g., I/O request status, thread coordination operations, and interrupt levels. Some operations modeled in this case study – for example, passing a key from one thread to another to allow the second thread to access the associated resource, and a lock model in which the proper key must be held to access the resource – are similar to features of Rust's ownership-based type system and channels for sending data between threads, which are distinct from the concept of typestate in Rust.

**Plaid.** Plaid is a general-purpose object-oriented programming language that treats states similarly to classes. Plaid is dynamically typed and typestate violations are reported as runtime errors (unlike Rust and Vault, in which typestate-related checks run at compile time). As a result, Plaid avoids some challenges with typestate checking in Rust, such as the issue of tracking typestate of objects in collections, at the cost of static guarantees.

Plaid provides several ways to interact with aliased objects. Objects may be defined as `shared`, meaning that there may be multiple mutable aliases to that object but no client is allowed to change the object's state (i.e., only modifications that do not involve a state transition are legal). The authors also propose the use of dynamic tests to check that an object's state has not been changed before some operations, which is more flexible than the former approach but introduces additional runtime overhead. Similar techniques could be achieved in Rust by using interior mutability, a pattern that provides a way to gain mutable access to an object while there exist immutable references to it. However, as in Plaid, this would give up the benefits of statically checking.

## 7 Conclusion

This article presents a new methodology for crash-consistent file system development. We propose the use of the typestate pattern in Rust to statically check crash-consistency invariants with low proof burden. We also introduce a novel crash-consistency mechanism, synchronous soft updates, that is well-suited to enforcement with the typestate pattern and that eliminates many challenges

associated with the original soft updates technique. We develop SquirrelFS, a new file system for persistent memory that uses statically-checked synchronous soft updates for crash consistency. SquirrelFS achieves comparable or better performance than other PM file systems and required no language modifications or verification expertise to build. SquirrelFS, its Alloy model, and our Alloy utilities are available at https://github.com/utsaslab/squirrelfs.

## Acknowledgment

We would like to thank the anonymous reviewers and the members of SaSLab and LASR at UT Austin for their insightful comments and feedback.

## References

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. 2009. Typestate-oriented programming. In *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications*. Association for Computing Machinery, New York, NY, USA, 1015–1022. DOI: https://doi.org/10.1145/1639950.1640073

[2] Sidney Amani, Alex Hixon, Zilin Chen, Christine Rizkallah, Peter Chubb, Liam O'Connor, Joel Beeren, Yutaka Nagashima, Japheth Lim, Thomas Sewell, et al. 2016. Cogent: Verifying high-assurance file system implementations. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 175–188. DOI: https://doi.org/10.1145/2872362.2872404

[3] Valerie Aurora. 2009. Soft updates, hard problems. Retrieved November 15, 2023 from https://lwn.net/Articles/339337/. (July 2009).

[4] Piotr Balcer. 2023. Exploring the Software Ecosystem for Compute Express Link (CXL) Memory. (May 2023). Retrieved November 15, 2023 from https://pmem.io/blog/2023/05/exploring-the-software-ecosystem-for-compute-express-link-cxl-memory/

[5] The Embedded Rust Book. 2024. Typestate Programming. (2024). Retrieved November 15, 2023 from https://docs.rust-embedded.org/book/static-guarantees/typestate-programming.html

[6] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, et al. 2021. Using lightweight formal methods to validate a key-value storage node in amazon S3. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 836–850.

[7] James Bornholt, Antoine Kaufmann, Jialin Li, Arvind Krishnamurthy, Emina Torlak, and Xi Wang. 2016. Specifying and checking file system crash-consistency models. In *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems*. Atlanta, GA, USA, 83–98.

[8] BTRFS developers. 2024. BTRFS Documentation. (2024). Retrieved November 15, 2023 from https://btrfs.readthedocs.io/en/latest/

[9] Tej Chajed, Joseph Tassarotti, Mark Theng, M. Frans Kaashoek, and Nickolai Zeldovich. 2022. Verifying the DaisyNFS concurrent and crash-safe file system with sequential reasoning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Carlsbad, CA, 447–463. Retrieved from https://www.usenix.org/conference/osdi22/presentation/chajed

[10] Haogang Chen, Tej Chajed, Alex Konradi, Stephanie Wang, Atalay İleri, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. 2017. Verifying a high-performance crash-safe file system using a tree specification. In *Proceedings of the 26th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 270–286. DOI: https://doi.org/10.1145/3132747.3132776

[11] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. 2015. Using crash hoare logic for certifying the FSCQ file system. In *Proceedings of the 25th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 18–37. DOI: https://doi.org/10.1145/2815400.2815402

[12] Vijay Chidambaram. 2015. *Orderless and Eventually Durable File Systems*. Ph.D. Dissertation. University of Wisconsin, Madison.

[13] Vijay Chidambaram, Tushar Sharma, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2012. Consistency without ordering. In *Proceedings of the 10th Conference on File and Storage Technologies*. San Jose, California, 101–116.

[14] CXL Consortium. 2023. Compute Express Link (CXL) specification. (2023). Retrieved November 15, 2023 from https://www.computeexpresslink.org/download-the-specification

[15] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 143–154. DOI: https://doi.org/10.1145/1807128.1807152

[16] Symas Corporation. 2024. LMDB. (2024). Retrieved November 15, 2023 from http://www.lmdb.tech

[17] Karl Crary, David Walker, and Greg Morrisett. 1999. Typed memory management in a calculus of capabilities. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, New York, NY, USA, 262–275. DOI: https://doi.org/10.1145/292540.292564

[18] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *Proceedings of the 2008 Tools and Algorithms for Construction and Analysis of Systems*. Springer, Berlin, 337–340. Retrieved from https://www.microsoft.com/en-us/research/publication/z3-an-efficient-smt-solver/

[19] Robert DeLine and Manuel Fähndrich. 2001. Enforcing high-level protocols in low-level software. In *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*. Association for Computing Machinery, New York, NY, USA, 59–69. DOI: https://doi.org/10.1145/378795.378811

[20] Mingkai Dong and Haibo Chen. 2017. Soft updates made simple and fast on non-volatile memory. In *Proceedings of the 2017 USENIX Annual Technical Conference*. USENIX Association, Santa Clara, CA, 719–731. Retrieved from https://www.usenix.org/conference/atc17/technical-sessions/presentation/dong

[21] Subramanya R. Dulloor, Sanjay Kumar, Anil Keshavamurthy, Philip Lantz, Dheeraj Reddy, Rajesh Sankaran, and Jeff Jackson. 2014. System software for persistent memory. In *Proceedings of the 9th European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, Article 15, 15 pages. DOI: https://doi.org/10.1145/2592798.2592814

[22] John Field, Deepak Goyal, G. Ramalingam, and Eran Yahav. 2005. Typestate verification: Abstraction techniques and complexity results. Springer Berlin. Retrieved November 15, 2023 from https://www.microsoft.com/en-us/research/publication/typestate-verification-abstraction-techniques-complexity-results/

[23] Rust for Linux. 2024. (2024). Retrieved November 15, 2023 from https://rust-for-linux.com/

[24] Christopher Frost, Mike Mammarella, Eddie Kohler, Andrew de los Reyes, Shant Hovsepian, Andrew Matsuoka, and Lei Zhang. 2007. Generalized file system dependencies. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 307–320. DOI: https://doi.org/10.1145/1294261.1294291

[25] Gregory R. Ganger and Yale N. Patt. 1994. Metadata Update Performance in File Systems. In *Proceedings of the 1st Symposium on Operating Systems Design and Implementation*. USENIX Association, Monterey, CA. Retrieved from https://www.usenix.org/conference/osdi-94/metadata-update-performance-file-systems

[26] Jon Gjenset. 2022. *Rust for Rustaceans*. No Starch Press.

[27] Robert B. Hagmann. 1987. Reimplementing the cedar file system using logging and group commit. In *Proceedings of the 11th ACM Symposium on Operating System Principles, Stouffer Austin Hotel, Austin, Texas, USA, November 8-11, 1987*. Les Belady (Ed.), ACM, 155–162. DOI: https://doi.org/10.1145/41457.37518

[28] Travis Hance, Andrea Lattuada, Chris Hawblitzel, Jon Howell, Rob Johnson, and Bryan Parno. 2020. Storage systems are distributed systems (so verify them that way!). In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. USENIX Association, USA, 17 pages.

[29] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jay Lorch, Bryan Parno, Justine Stephenson, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving practical distributed systems correct. In *Proceedings of the ACM Symposium on Operating Systems Principles*. ACM - Association for Computing Machinery. Retrieved from https://www.microsoft.com/en-us/research/publication/ironfleet-proving-practical-distributed-systems-correct/

[30] Dave Hitz, James Lau, and Michael A. Malcolm. 1994. File system design for an NFS file server appliance. In *USENIX Winter 1994 Technical Conference, San Francisco, California, USA, January 17-21, 1994, Conference Proceedings*. USENIX Association, 235–246. Retrieved from https://www.usenix.org/conference/usenix-winter-1994-technical-conference/file-system-design-nfs-file-server-appliance

[31] Morteza Hoseinzadeh and Steven Swanson. 2021. Corundum: Statically-enforced persistent memory safety. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 429–442. DOI: https://doi.org/10.1145/3445814.3446710

[32] Daniel Jackson. 2016. *Software Abstractions*. MIT.

[33] Rohan Kadekodi, Saurabh Kadekodi, Soujanya Ponnapalli, Harshad Shirwadkar, Gregory R. Ganger, Aasheesh Kolli, and Vijay Chidambaram. 2021. WineFS: A hugepage-aware file system for persistent memory that ages gracefully. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 804–818. DOI: https://doi.org/10.1145/3477132.3483567

[34] Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. 2019. SplitFS: Reducing software overhead in file systems for persistent memory. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 494–508. DOI: https://doi.org/10.1145/3341301.3359631

[35] Samuel Kalbfleisch, Lukas Werling, and Frank Bellosa. 2022. Vinter: Automatic non-volatile memory crash consistency testing for full systems. In *Proceedings of the 2022 USENIX Annual Technical Conference*. USENIX Association, Carlsbad, CA, 933–950. Retrieved from https://www.usenix.org/conference/atc22/presentation/werling

[36] Seulbae Kim, Meng Xu, Sanidhya Kashyap, Jungyeon Yoon, Wen Xu, and Taesoo Kim. 2019. Finding semantic bugs in file systems with an extensible fuzzing framework. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 147–161. DOI: https://doi.org/10.1145/3341301.3359662

[37] Seulbae Kim, Meng Xu, Sanidhya Kashyap, Jungyeon Yoon, Wen Xu, and Taesoo Kim. 2020. Finding bugs in file systems with an extensible fuzzing framework. *ACM Transactions on Storage* 16, 2 (2020), 35 pages. DOI: https://doi.org/10.1145/3391202

[38] Steve Klabnik and Carol Nichols. 2018. *The Rust Programming Language*. No Starch Press, USA.

[39] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. 2017. Strata: A cross media file system. In *Proceedings of the 26th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 460–477. DOI: https://doi.org/10.1145/3132747.3132770

[40] William Landi. 1992. Undecidability of static analysis. *ACM Letters on Programming Languages and Systems* 1, 4 (1992), 323–337. DOI: https://doi.org/10.1145/161494.161501

[41] Ubuntu Bugs LaunchPad. 2018. Bug #317781: Ext4 Data Loss. Retrieved November 15, 2023 from https://bugs.launchpad.net/ubuntu/+source/linux/+bug/317781?comments=all. (2018).

[42] Hayley LeBlanc, Shankara Pailoor, Om Saran K. R. E., Isil Dillig, James Bornholt, and Vijay Chidambaram. 2023. Chipmunk: Investigating crash-consistency in persistent-memory file systems. In *Proceedings of the 18th European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 718–733. DOI: https://doi.org/10.1145/3552326.3567498

[43] Linux developers. 2024. Direct Access for files. (2024). Retrieved November 15, 2023 from https://www.kernel.org/doc/Documentation/filesystems/dax.txt

[44] Linux Test Project developers. 2024. Linux Test Project. (2024). Retrieved November 15, 2023 from https://linux-test-project.github.io/

[45] R. Lorie. 1977. Physical integrity in a large segmented database. *ACM Transactions on Databases* 2, 1 (1977), 91–104.

[46] Marshall Kirk McKusick and Gregory R. Ganger. 1999. Soft updates: A technique for eliminating most synchronous writes in the fast filesystem. In *Proceedings of the 1999 USENIX Annual Technical Conference*. USENIX Association, Monterey, CA. Retrieved from https://www.usenix.org/conference/1999-usenix-annual-technical-conference/soft-updates-technique-eliminating-most

[47] Marshall Kirk McKusick, George Neville-Neil, and Robert N. M. Watson. 2014. *The Design and Implementation of the FreeBSD Operating System* (2nd. ed.). Addison-Wesley Professional.

[48] Samantha Miller, Kaiyuan Zhang, Mengqi Chen, Ryan Jennings, Ang Chen, Danyang Zhuo, and Thomas Anderson. 2021. High velocity kernel file systems with bento. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies*. USENIX Association, 65–79. Retrieved from https://www.usenix.org/conference/fast21/presentation/miller

[49] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, and Vijay Chidambaram. 2019. CrashMonkey and ACE: Systematically testing file-system crash consistency. *ACM Transactions on Storage* 15, 2 (2019), 34 pages. DOI: https://doi.org/10.1145/3320275

[50] Ian Neal, Ben Reeves, Ben Stoler, Andrew Quinn, Youngjin Kwon, Simon Peter, and Baris Kasikci. 2020. AGAMOTTO: How persistent is your persistent memory application?. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 1047–1064. Retrieved from https://www.usenix.org/conference/osdi20/presentation/neal

[51] Roger M. Needham, David K. Gifford, and Mike Schroeder. 1988. The cedar file system. *Communications of the ACM* 31, 3 (1988), 288–298. https://dl.acm.org/doi/10.1145/42392.42398

[52] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. 2015. How amazon web services uses formal methods. *Communications of the ACM* 58, 5 (2015), 66–73. https://dl.acm.org/doi/10.1145/2699417

[53] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2015. Crash consistency. *Communications of the ACM* 58, 10 (2015), 46–51. DOI: https://doi.org/10.1145/2788401

[54] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2014. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation, Broomfield, CO, USA, October 6-8, 2014*. Jason Flinn and Hank Levy (Eds.), USENIX Association, 433–448. Retrieved from https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[55] RocksDB developers. 2024. RocksDB. (2024). Retrieved November 15, 2023 from https://rocksdb.org/

[56] Andy Rudoff. 2017. Persistent Memory Programming. *;login:* 42, 2 (2017), 34–40. Retrieved from https://www.usenix.org/system/files/login/articles/login_summer17_07_rudoff.pdf

[57] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2016. Push-button verification of file systems via crash refinement. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Savannah, GA, 1–16. Retrieved from https://www.usenix.org/conference/osdi16/technical-sessions/presentation/sigurbjarnarson

[58] Robert E. Strom and Shaula Yemini. 1986. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. DOI: https://doi.org/10.1109/TSE.1986.6312929

[59] Joshua Sunshine, Karl Naden, Sven Stork, Jonathan Aldrich, and Éric Tanter. 2011. First-class state change in plaid. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications*. Association for Computing Machinery, New York, NY, USA, 713–732. DOI: https://doi.org/10.1145/2048066.2048122

[60] Vasily Tarasov, Erez Zadok, and Spencer Shepler. 2016. Filebench: A Flexible Framework for File System Benchmarking. *;login:* 41, 1 (2016), 6–12. Retrieved from https://www.usenix.org/publications/login/spring2016/tarasov

[61] The Coq Development Team. 2023. The Coq Proof Assistant. (June 2023). https://github.com/rocq-prover/rocq/wiki/Publishing-Tools#how-can-i-cite-the-coq-reference-manual

[62] Jacob Van Geffen, Xi Wang, Emina Torlak, and James Bornholt. 2023. Synthesis-aided crash consistency for storage systems. In *Proceedings of the 37th European Conference on Object-Oriented Programming (* Karim Ali and Guido Salvaneschi (Eds.), Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 35:1–35:26. DOI: https://doi.org/10.4230/LIPIcs.ECOOP.2023.35

[63] xfstests developers. 2024. xfstests. (2024). Retrieved November 15, 2023 from https://github.com/kdave/xfstests

[64] Jian Xu, Juno Kim, Amir Saman Memaripour, and Steven Swanson. 2019. Finding and fixing performance pathologies in persistent memory software stacks. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems, Providence, RI, USA, April 13-17, 2019*. Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck (Eds.), ACM, 427–439. DOI: https://doi.org/10.1145/3297858.3304077

[65] Jian Xu and Steven Swanson. 2016. NOVA: A Log-structured File System for Hybrid Volatile/Non-volatile Main Memories. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies*. USENIX Association, Santa Clara, CA, 323–338. Retrieved from https://www.usenix.org/conference/fast16/technical-sessions/presentation/xu

[66] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steven Swanson. 2020. An empirical guide to the behavior and use of scalable persistent memory. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies, FAST 2020, Santa Clara, CA, USA, February 24-27, 2020*. Sam H. Noh and Brent Welch (Eds.), USENIX Association, 169–182. Retrieved from https://www.usenix.org/conference/fast20/presentation/yang

[67] Junfeng Yang, Paul Twohey, Dawson Engler, and Madanlal Musuvathi. 2004. Using model checking to find serious file system errors. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*. USENIX Association, USA, 273–287.

[68] Diyu Zhou, Vojtech Aschenbrenner, Tao Lyu, Jian Zhang, Sudarsun Kannan, and Sanidhya Kashyap. 2023. Enabling high-performance and secure userspace NVM file systems with the trio architecture. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 150–165. DOI: https://doi.org/10.1145/3600006.3613171

Received 30 September 2024; revised 24 February 2025; accepted 13 June 2025